



Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

Masterarbeit
Studiengang: Informatik

JSONPath-Schnittstelle für NoSQL-Datenbanken

von

Paul Mayr

77868

Betreuender Professor: Prof. Dr. Winfried Bantel
Zweitprüfer: Prof. Dr. habil. Christian Heinlein

Einreichungsdatum: 01. Juni 2026

Eidesstattliche Erklärung

Hiermit erkläre ich, **Paul Mayr**, dass ich die vorliegenden Angaben in dieser Arbeit wahrheitsgetreu und selbständig verfasst habe.

Weiterhin versichere ich, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben, dass alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und dass die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Ort, Datum

Unterschrift (Student)

Vorwort

Die vorliegende Masterthesis wurde als Abschluss meines Studiums an der Hochschule Aalen im Masterstudiengang Informatik mit Schwerpunkt Medieninformatik verfasst. Die Arbeit wurde zwischen dem 01. Dezember 2025 und dem 01. Juni 2026 unter der Betreuung von Herrn Prof. Dr. Winfried Bantel angefertigt.

Mein besonderer Dank gilt Herrn Prof. Dr. Bantel für sein fachliches Feedback sowie die freundliche und zuverlässige Unterstützung und Betreuung in allen Phasen dieser Arbeit.

Paul Mayr

Heidenheim, 1. Juni 2026

Kurzfassung

In dieser Arbeit wird die Entwicklung eines *JSONPath*-Compilers nach RFC-9535 mit *flex* und *Bison* vorgestellt. Diese erfolgt als erster Teil der Erfüllung des Gesamtziels, einen Interpreter für *JSONPath*-Ausdrücke zu entwickeln, damit diese für Abfragen an die NoSQL-Datenbank *YottaDB* verwendet werden können.

Der Fokus des entwickelten Systems liegt dabei neben einer klar strukturierten komponentenbasierten Architektur auf einer Implementation, die zur *JSONPath*-Definition in RFC-9535 konform ist.

Der vom Compiler mittels der *Bison-Syntaxtree-Hack*-Klasse automatisiert erzeugte Syntaxbaum stellt neben dem modularen Gesamtsystem den Ausgangspunkt für die Entwicklung eines *JSONPath*-Interpreters dar, für welchen auch ein theoretischer Grundstein in dieser Arbeit gelegt wird.

Diese Arbeit befasst sich außerdem ebenfalls theoretisch mit Möglichkeiten, die Suche in *YottaDB* beschleunigen zu können und erstellt hierfür Kriterien, anhand welcher mögliche Algorithmen auf deren Eignung in Hinsicht auf Laufzeit und Speicherkomplexität überprüft werden können.

Inhaltsverzeichnis

Eidesstattliche Erklärung	i
Vorwort	ii
Kurzfassung	iii
Abbildungsverzeichnis	vii
Abkürzungsverzeichnis	viii
1. Einleitung	1
1.1. Motivation	1
1.2. Problemstellung und -abgrenzung	2
1.3. Ziel der Arbeit	2
1.4. Vorgehen	3
2. Grundlagen	5
2.1. JSON	5
2.2. JSONPath	6
2.3. NoSQL-Datenbanken	6
2.4. YottaDB	7
2.5. Phasen von Compilern	8
2.5.1. Lexikalische Analyse	9
2.5.2. Syntaktische Analyse	9
2.6. flex	10
2.7. Bison	10
2.7.1. GLR	10
3. Problemanalyse	12
3.1. JSONPath-Funktionsumfang	13
3.1.1. Allgemein	13
3.1.2. Wurzel	13
3.1.3. Segmente	13
3.1.4. Selektoren	15
3.1.5. Whitespaces	19
3.1.6. Normalisierte JSONPath-Ausdrücke	21
3.1.7. Erlaubte Eingangszeichen	22

3.1.8. Fehlerbehandlung	23
3.2. Aufbau des Compilers	23
3.2.1. Lexikalische Analyse	23
3.2.2. Syntaktische Analyse	24
3.2.3. Zwischencode	24
3.3. Aufbau des Interpreters	25
3.4. Beschleunigung der Suche in YottaDB	25
3.4.1. Laufzeit von Suchen nach exakter Übereinstimmung	26
3.4.2. Speicherkomplexität von Suchen nach exakter Übereinstimmung	26
3.4.3. Parametrisierung der Suchalgorithmen	27
3.4.4. Zusammenfassung	27
3.5. Anforderungsverzeichnis	28
4. Lösungskonzept	30
4.1. Gesamtsystem	30
4.1.1. Systemarchitektur	31
4.1.2. Kompetenzverteilung Scanner / Parser	33
4.1.3. Anpassungen der Grammatik	34
4.1.4. Technologie-Auswahl	35
4.2. JSONPath-Grammatik in EBNF	37
4.3. Controller	38
4.4. flex-Scanner	39
4.4.1. Whitespace-Verarbeitung	39
4.5. Bison-Parser	41
4.5.1. Wiederholungsschema	41
4.5.2. Optionsschema	42
4.5.3. Epsilon-Ableitungen	42
4.5.4. Normalisierte Ausdrücke	43
4.5.5. Bison-Syntaxtree-Hack	43
4.6. Compiler-Tests	44
4.7. Interpreter	44
4.8. Beschleunigte Suche in YottaDB	45
4.8.1. Vollständige Zerlegung einer Schlüsselebene	46
4.8.2. Anpassung an verbesserte Laufzeit	48
4.8.3. Reduktion des Zeichensatzes	49
4.8.4. Reduktion der Schlüssellänge	50
5. Implementierung	52
5.1. Gesamtsystem	52
5.2. Controller	52
5.3. flex-Scanner	53
5.3.1. Whitespaces	54
5.3.2. Schlüsselworte und Operatoren	55
5.3.3. Komplexe reguläre Ausdrücke	55

5.3.4. Startbedingungen	56
5.4. Bison-Parser	59
5.4.1. Normalisierte Ausdrücke	59
5.4.2. Mehrdeutigkeiten in der Grammatik	60
5.5. Automatisierte Syntaxbaumerstellung mit Bison-Syntaxtree-Hack . .	61
5.5.1. Visualisierung	61
6. Evaluierung	62
6.1. Erreichte Ergebnisse anhand Anforderungsliste	62
6.1.1. JSONPath-Funktionsumfang	62
6.1.2. JSONPath-Compiler	63
6.1.3. JSONPath-Interpreter	64
6.1.4. Beschleunigen der Suche in YottaDB	64
6.2. RFC-Konformität	64
6.3. Automatisierte Tests	65
6.4. Performance	65
7. Zusammenfassung und Ausblick	66
7.1. Erreichte Ergebnisse	66
7.2. Ausblick	67
Literatur	68
A. JSONPath-Grammatik in EBNF	69
A.1. Grammatik-Regeln	69
A.2. Zeichenregeln	71
A.3. Normalisierte Abfragen	73
B. Syntaxbäume	74

Abbildungsverzeichnis

2.1. Phasen eines Compilers (vereinfacht nach [6] [2])	8
4.1. Architekturübersicht des Gesamtsystems	30
4.2. Architekturdiagramm im Detail	31
5.1. Zustandsdiagramm der Startbedingungen	56
B.1. Syntaxbaum für Ausdruck $[a']$	74
B.2. Syntaxbaum für Ausdruck $a[1]$	74
B.3. Syntaxbaum für Ausdruck $[?@.a > 1 \ \&\& \ @.b \leq 0]$	75

Abkürzungsverzeichnis

ABNF	<i>Angereicherte Backus-Naur-Form</i>	9
BNF	<i>Backus-Naur-Form</i>	10
EBNF	<i>Erweiterte Backus-Naur-Form</i>	37
GLR	<i>Generalized LR parser</i>	11
GT.M	<i>Greystone Technology Mumps</i>	7
JSON	<i>JavaScript Object Notation</i>	1
RR	<i>Reduziere / Reduziere</i>	10
SR	<i>Schiebe / Reduziere</i>	10
XML	<i>Extensible Markup Language</i>	1

1. Einleitung

Seit dem Turmbau zu Babel, bei welchem - so die Geschichte - zum ersten Mal verschiedene Sprachen eingeführt wurden, stellen Übersetzer eine nicht wegzudenkende Rolle in der Kommunikation dar.

Nicht nur die Kommunikation zwischen Menschen, sondern auch die Kommunikation zwischen Systemen erfordert oft die Übersetzung von einer Sprache in eine andere, auch wenn die Übersetzung für die Kommunikation zwischen Systemen meist etwas einfacher ist, da ihre Sprachen sich in festen Regeln bewegen.

1.1. Motivation

Das *JavaScript Object Notation* (JSON)-Datenaustauschformat hat sich vielerorts als Ersatz für *Extensible Markup Language* (XML) für die Übertragung von Daten beziehungsweise die Kommunikation zwischen Systemen eingebürgert.

Bereits 2007 wurde hierfür, angelehnt an die pfadbasierte XML-Abfragesprache *XPath*, die Abfragesprache *JSONPath* vorgeschlagen. Erst 17 Jahre später wurde diese 2024 als *RFC-9535* normiert. [8]

Trotz der zunehmend weiten Verbreitung von JSON und JSONPath ist deren Verwendung an einigen Stellen noch nicht unterstützt. Eine solche Stelle ist die *NoSQL*-Datenbank *YottaDB*, welche aufgrund der intern verwendeten Objektstrukturen für die gespeicherten Daten dem Aufbau von JSON-Daten ähnelt.

Die Möglichkeit, JSONPath-Abfragen für NoSQL-Datenobjekte wie die von YottaDB verwenden zu können, wäre also eine naheliegende Vereinfachung im Umgang mit diesen Daten.

1.2. Problemstellung und -abgrenzung

In dieser Arbeit wird ein System entwickelt, welches Abfragen auf die NoSQL-Datenbank YottaDB in Form von JSONPath-Abfragen nach RFC-9535 ermöglichen soll.

Das System soll dafür eingehende JSONPath-Abfragen zunächst validieren und fehlerhafte Abfragen zurückweisen, um anschließend die validen Abfragen in Syntaxbäume zu überführen und anhand dieser die Abfragen unter Verwendung eines in YottaDB gespeicherten Datensatzes auszuwerten.

Das Zielsystem kann also aufgrund seiner Funktionsweise als ein *Compiler* und *Interpreter* für JSONPath zu YottaDB betrachtet werden.

Um Abfragen auf großen Datensätzen performant zu halten, soll das System den Datensatz beispielsweise über geeignete Indizierung erweitern, um schnellere Suchen zu ermöglichen.

Die Erzeugung und Validierung der Datensätze, gegen welche das System eingehende Abfragen auswerten soll, sind nicht Teil dieser Arbeit.

Ebenso gliedert sich das System in keine bestehenden Arbeitsabläufe ein und entspricht daher keinen vordefinierten Schnittstellen.

Innerhalb dieser Arbeit wird das System als alleinstehendes Konsolenprogramm entwickelt; eine Weiterentwicklung des Systems beispielsweise für die Einbindung in einen Arbeitsablauf soll jedoch durch klar abgetrennte Komponenten und übersichtliche interne Schnittstellen ermöglicht werden.

1.3. Ziel der Arbeit

Nachfolgend werden die Ziele dieser Arbeit definiert und anschließend formalisiert zusammengefasst.

Ziel dieser Arbeit ist die Entwicklung einer zu RFC-9535 konformen Anwendung zur Auswertung von JSONPath-Abfragen auf in einer YottaDB NoSQL-Datenbank gespeicherte Datenobjekte. Die Anwendung soll dabei den vollständigen Funktionsumfang von JSONPath abbilden können, wie dieser in RFC-9535 definiert wird.

Hierfür muss die entwickelte Anwendung Eingaben anhand der Anforderungen von RFC-9535 überprüfen und Abfragen, die keine wohlgeformten und validen JSONPath-Abfragen darstellen, verwerfen.

Das System muss die validen JSONPath-Abfragen anschließend gegen das gegebene Datenobjekt auswerten und das Ergebnis der Auswertung zurückgeben. Für diese Auswertung sollen die Abfragen zunächst in Syntaxbäume überführt werden, anhand welcher die Auswertung stattfinden kann.

Damit das System auch bei größeren Datenmengen performante Auswertung von JSONPath-Abfragen erbringen kann, soll über geeignete Indizierung der Datensätze die Suche innerhalb dieser beschleunigt werden.

Um die Anpassbarkeit, Wartbarkeit und Erweiterbarkeit des Systems für weitere Arbeiten zu ermöglichen, soll dieses sich in klar getrennte Komponenten unterteilen. Außerdem sollen die Komponenten durch eine einfache Dokumentation leicht nachvollziehbar gemacht werden.

Zusammengefasst sind die Ziele dieser Arbeit also:

- *Primärziel P*: Entwicklung einer Anwendung zur Auswertung von JSONPath-Abfragen auf YottaDB-Datenobjekten
- *Sekundärziel S1*: Vollständige Umsetzung des JSONPath-Funktionsumfangs nach RFC-9535
- *Sekundärziel S2*: Überprüfung der Eingaben auf Konformität zu RFC-9535
- *Sekundärziel S3*: Überführung der Abfragen in Syntaxbäume
- *Sekundärziel S4*: Auswertung der Syntaxbäume gegen die Datenobjekte
- *Sekundärziel S5*: Beschleunigen der Auswertung durch Indizierung
- *Sekundärziel S6*: Komponentenbasierte Systemarchitektur für zukünftige Erweiterungen

1.4. Vorgehen

Für die Erreichung der in Abschnitt 1.3 genannten Ziele werden zunächst die verwendeten Technologien und die für das Verständnis dieser Arbeit relevanten Konzepte vorgestellt.

In der darauf folgenden Problemanalyse werden die Ziele genauer untersucht und daraus entstehende Anforderungen ermittelt, welche in einer Anforderungsliste zusammengefasst werden.

Ausgehend von dieser Anforderungsliste wird ein Lösungskonzept erarbeitet und verschiedene Entscheidungen und alternative Ansätze werden betrachtet und abgewogen.

Daraufhin wird die Umsetzung der im Lösungskonzept erarbeiteten Ansätze dargestellt und einzelne Implementationsdetails werden illustrativ herausgegriffen und genauer betrachtet.

Die Ergebnisse werden anschließend anhand der definierten Ziele und Anforderungen evaluiert und bewertet, inwiefern diese den Zielen und Anforderungen gerecht werden.

Zuletzt werden die erreichten Ergebnisse nochmals zusammengefasst und ein Ausblick auf mögliche Ansätze zur Übertragung und Erweiterung dieser Ergebnisse gegeben.

2. Grundlagen

Im Folgenden werden die für das Verständnis dieser Arbeit relevanten Technologien und Konzepte vorgestellt. Ziel hiervon ist nicht eine erschöpfende Definition aller verwendeten Begriffe, sondern vielmehr eine Übersicht und oberflächliche Klärung einiger relevanter Begriffe.

Eine ausführlichere Betrachtung von JSONPath, insbesondere dem in RFC-9535 definierten Funktionsumfang dieser Abfragesprache, erfolgt in Kapitel 3, da diese stark prägend für die ermittelten Anforderungen ist.

2.1. JSON

JSON ist ein einfaches, text-basiertes und sprachunabhängiges Austauschformat für strukturierte Daten, welches in RFC-8259 vereinheitlicht wurde. JSON-Daten beziehungsweise -Objekte können als eine Baumstruktur aus Schlüssel-Wert-Paaren betrachtet werden, wobei der Wurzelknoten der Knoten ist, dessen Wert das gesamte JSON-Objekt ausmacht. [10]

Der Schlüssel eines Schlüssel-Wert-Paars, der auch als Name des Paars bezeichnet wird, ist vom primitiven Typ *String*. Der Wert des Paars kann von einem der primitiven Typen *String*, *Zahl*, *Boolean* und *null* oder einem der komplexen Typen *Objekt* und *Array* sein.

Ein JSON-Objekt ist eine ungeordnete Sammlung beliebig vieler Schlüssel-Wert-Paare. Jedes JSON-Objekt kann einzeln betrachtet auch der Wert des Wurzelknotens eines Baums sein, der das Objekt abbildet.

Ein Array ist eine geordnete Sequenz von beliebig vielen Werten. Diese Werte haben selbst keinen zugeordneten Namen, können aber wiederum Objekte mit Namen sein beziehungsweise beinhalten.

2.2. JSONPath

Inspiziert von der pfadbasierten XML-Abfragesprache *XPath* stellt *JSONPath* nach RFC-9535 eine String-Syntax für die Auswahl und Extraktion von JSON-Werten aus einem gegebenen JSON-Objekt dar.

JSONPath-Ausdrücke werden auf valide JSON-Objekte angewendet, um daraus Werte oder Knoten auszuwählen. Das Ergebnis einer JSONPath-Abfrage ist eine Knotenliste, welche im Falle, dass die Abfrage zu einem oder mehreren Knoten im JSON-Objekt führt, diese Knoten enthält.^[8]

JSONPath-Ausdrücke starten vom Wurzelknoten des JSON-Objekts und definieren über eine beliebige Anzahl von Segmenten, welche Kind- oder Nachfolgeknoten auf der jeweiligen Ebene des JSON-Objekts für die weitere Verarbeitung auszuwählen sind.

```
1 $.store.book
```

So werden zum Beispiel im obigen JSONPath-Ausdruck alle Kindknoten des Wurzelknotens *\$* ausgewählt, die *store* heißen, und von diesen wiederum alle Kindknoten, die *book* heißen. Konkret bedeutet das, dass alle Bücher der im JSON-Objekt aufgeführten Läden aufgelistet werden sollen.

2.3. NoSQL-Datenbanken

NoSQL-Datenbanken oder auch *Not only SQL*-Datenbanken unterscheiden sich von den relationalen Datenbanken durch die Form ihres Datenspeichers.

Die wichtigsten Typen von NoSQL-Datenbanken sind *dokumentbasiert*, *Schlüssel-Wert-basiert*, *spaltenübergreifend* und *Graphdatenbanken*. Sie unterscheiden sich primär in den verwendeten Datenmodellen, alle bieten jedoch flexible Schemata zur Datenspeicherung an und sind meist gut für große Datenmengen skalierbar.^[9]

2.4. YottaDB

YottaDB ist eine 2017 aus *Greystone Technology Mumps* (GT.M) hervorgegangene NoSQL-Datenbank, die Daten als Schlüssel-Wert-Tupel abspeichert. Jedes Schlüssel-Wert- n -Tupel wird als ein Knoten betrachtet. Die ersten $n - 1$ Elemente dieser n -Tupel sind die Schlüssel und das letzte Element ist der Wert.

```
1  ["Capital", "Belgium", "Brussels"]
2  ["Capital", "Thailand", "Bangkok"]
```

Wie das obige Beispiel illustriert, sind die Schlüssel in einer Art hierarchischer Sortierung, in welcher der erste Schlüssel (in diesem Fall *Capital*) den Inhaltstyp des Wertes vorgibt und die weiteren Schlüssel den Wert eindeutig identifizieren. Der erste Schlüssel wird in YottaDB als *Variable* bezeichnet, unter welchem sich die anderen Schlüssel einordnen.

Die Menge aller Knoten unter einer Variablen können als Baum betrachtet werden, dessen Wurzel die Variable selbst ist. In YottaDB-Bäumen können auch Knoten mit Werten Nachfahren haben, wodurch sich YottaDB-Bäume von JSON unterscheiden. Alle JSON-Strukturen können in YottaDB-Bäumen abgebildet werden, jedoch können nicht alle YottaDB-Bäume in JSON-Strukturen abgebildet werden.

Da YottaDB kein Schema für die zu speichernden Daten vorgibt, kann für Kompatibilität zu JSON ein Schema verwendet werden, in welchem Knoten entweder Werte oder Nachfolger haben können.[\[11\]](#) [\[12\]](#)

2.5. Phasen von Compilern

Ein Compiler übersetzt Code von seiner Ausgangssprache in eine andere formale Sprache. Um Compiler möglichst flexibel und wartbar aufzubauen, werden diese im Normalfall in verschiedene Phasen unterteilt.

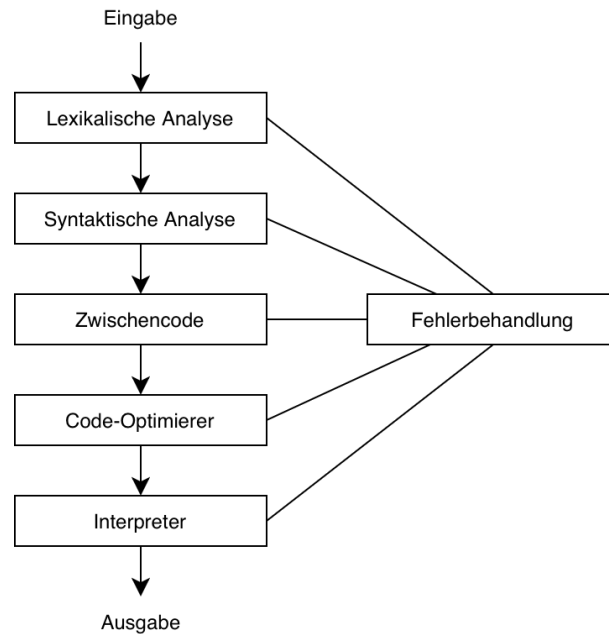


Abbildung 2.1.: Phasen eines Compilers (vereinfacht nach [6] [2])

Abbildung 2.1 stellt eine vereinfachte Form der Compiler-Phasen dar, in welcher das Ergebnis nicht die Zielsprache, sondern das Ergebnis eines Interpreters ist. Je nach Einsatzbereich beziehungsweise Implementation können sich diese Phasen unterscheiden, durch weitere ergänzt werden oder teilweise zusammengefasst werden, jedoch bieten sie einen sinnvollen Anhaltspunkt für den Aufbau eines Compilers.[6] [2]

Da der Schwerpunkt dieser Arbeit auf der lexikalischen und syntaktischen Analyse liegt, werden diese Phasen im Folgenden etwas genauer hervorgehoben.

2.5.1. Lexikalische Analyse

In der *lexikalischen Analyse* wird die Eingabe in die von der Grammatik erwarteten Tokens zerlegt. Die genaue Aufteilung beziehungsweise Definition, welche Eingabeteile in welche Tokens übersetzt werden, muss abhängig von der Grammatik und den Anforderungen an das System passend gewählt werden. [6] [2]

Die Unterteilung der Eingabe erfolgt anhand von Regeln, die zum Beispiel die Form von regulären Ausdrücken annehmen können.

Ein Programm, welches eine lexikalische Analyse eines Eingabetextes durchführt, wird *Scanner* genannt.

2.5.2. Syntaktische Analyse

Die *syntaktische Analyse* untersucht die Eingabe, beziehungsweise die aus der lexikalischen Analyse hervorgehenden Tokens, anhand einer vorgegebenen Grammatik, ob diese als Syntax aus der Grammatik hervorgehen können. [6] [2]

Für die Angabe der Grammatik wird oft eine Art der Backus-Naur-Form wie die *Angereicherte Backus-Naur-Form* (ABNF) oder die *ebnf* verwendet. Diese definiert Produktionsregeln, mit welchen nichtterminale Symbole zu einer Folge von terminalen oder nichtterminalen Symbolen abgebildet werden können.

Die syntaktische Analyse versucht, für die in der Eingabe gefundenen Tokens eine Folge von Produktionsregeln zu finden, durch welche die Eingabe entstehen kann. Wird keine solche Folge gefunden, lässt sich die Eingabe nicht aus der Grammatik herleiten und wird verworfen.

Ein Programm, welches eine syntaktische Analyse durchführt, wird *Parser* genannt.

2.6. flex

flex, oder *the fast lexical analyzer generator*, ist ein Generatortool, welches anhand von Regeln bestehend aus regulären Ausdrücken und C-Code einen Scanner generieren kann. [3] [7]

Die Regeln, aus denen der Scanner erzeugt werden soll, müssen in der sogenannten flex-Datei in der vorgegebenen Struktur aufgeschrieben werden.

Die regulären Ausdrücke werden in der Reihenfolge, in der sie in der Datei stehen, auf den Eingabetext gematcht und entweder das längste gefundene Match oder im Falle von mehreren längsten Matches das in der Datei zuerst geschriebene Match wird gewählt.

Anschließend wird der zum Match gehörende C-Code ausgeführt und zum Beispiel der passende Token zurückgegeben.[6] [2]

2.7. Bison

Bison ist ein Generatortool, welches aus einer kontextfreien Grammatik einen deterministischen oder allgemeinen LR-Parser erzeugt.

Ein LR-Parser ist ein sogenannter *Bottom-Up*-Parser für LR-Grammatiken. Dabei wird ausgehend von jeweils einem Blatt des Syntaxbaums versucht, den Eingabetext auf das Startsymbol zu reduzieren, indem die Produktionsregeln der Grammatik rückwärts angewendet werden, was *Reduzieren* genannt wird. [2] [4]

Eine Bison-Datei enthält Grammatik-Regeln in einer Form ähnlich der *Backus-Naur-Form* (BNF) und zugehörige C-Code-Anweisungen, sogenannten *semantischen Aktionen*, die beim Reduzieren der Regel ausgeführt werden sollen.

2.7.1. GLR

Mehrdeutigkeiten in der zu parsenden Grammatik können von einem LR-Parser nicht aufgelöst werden und führen zu *Schiebe / Reduziere* (SR)- und *Reduziere / Reduziere* (RR)-Konflikten.

Ein deterministischer LR-Parser kann in einem solchen Konfliktfall zu einem falschen Ergebnis führen, da standardmäßig die erste Option ausprobiert wird und deren Erfolg über den Erfolg des Parsens entscheidet.

Eine mögliche Lösung, eine mehrdeutige Grammatik zu parsen, ist die Verwendung eines allgemeinen LR-Parsers (englisch *Generalized LR parser* (GLR)).

GLR-Parser behandeln Grammatiken ohne Mehrdeutigkeiten identisch zu deterministischen LR-Parsern, können aber im Konfliktfall beide Optionen weiterverfolgen. Dabei klonet sich der Parser effektiv und beide Klone arbeiten parallel die Eingabe weiter ab.

Bei einem weiteren Konflikt klonet sich der Parser an dieser Stelle erneut, sodass zu jeder Zeit beliebig viele Alternativen erkundet werden.

Läuft ein Parserklon in einen Fehler, hört dessen Parsevorgang auf und die anderen Klone werden weiterverfolgt. Wenn alle Parserklone in einen Fehler gelaufen sind, ist der Parsevorgang fehlgeschlagen. Parserklone können auch wieder zusammenlaufen, wenn sie die Eingabe auf eine identische Symbolmenge reduziert haben.

Solange mehr als eine Instanz des Parsers existiert, werden semantische Aktionen nicht ausgeführt und in der jeweiligen Parserinstanz gespeichert. Wenn zwei Parserklone zusammengeführt werden, werden die gespeicherten semantischen Aktionen beider Klone in der übrigbleibenden Instanz gespeichert.

Wenn alle Klone zusammengeführt wurden, sodass nur noch eine Parserinstanz existiert, werden die ausstehenden semantischen Aktionen entweder nach Präzedenz der involvierten Grammatikregeln aufgelöst oder alle Aktionen werden ausgeführt und anhand einer nutzerdefinierten Funktion zu einem Ergebnis zusammengeführt.

[5]

3. Problemanalyse

Im Folgenden werden die in Abschnitt 1.3 definierten Ziele genauer betrachtet und die daraus entstehenden Anforderungen für diese Arbeit beziehungsweise das in dieser Arbeit zu entwickelnde System erörtert. Die ermittelten Anforderungen werden anschließend in ein Anforderungsverzeichnis überführt, anhand dessen sich die restliche Arbeit orientieren und strukturieren kann.

Das Primärziel P erfordert auf oberster Ebene, dass das System die Rolle eines sogenannten *Interpreters* erfüllt, also die Auswertung einer in einer formalen Sprache notierten Eingabe durchführt.

Um die Anwendung im Sinne von der in Sekundärziel S6 beschriebenen Erweiterbarkeit möglichst flexibel zu halten, bietet es sich jedoch an, dem Interpreter einen *Compiler* vorzusetzen, welcher die Eingabeüberprüfung übernimmt und die Eingabe in einen geeigneten Zwischencode überführt.

Dadurch lassen sich Eingabevalidierung und -auswertung voneinander isoliert behandeln und durch eventuelle Optimierungen am Zwischencode lässt sich auch die Auswertung vereinfachen.

In dieser Aufteilung ist der Compiler für die Realisierung von Sekundärzielen S1, S2 und S3 verantwortlich, während der Interpreter indirekt für Sekundärziel S1 und direkt für Sekundärziele S4 und S5 verantwortlich ist.

Mit anderen Worten bedeutet das, dass der Compiler den vollständigen JSONPath-Funktionsumfang nach RFC-9535 in Zwischencode überführen können muss und dabei die Eingabe auf Wohlgeformtheit und Validität untersucht, während der Interpreter inhaltlich die JSONPath-Funktionen aus dem Zwischencode auf den YottaDB-Datenobjekten auswerten muss und auch für eventuelle Beschleunigungen für große Datensätze verantwortlich ist.

Compiler und Interpreter sind dabei an dieser Stelle nur vorübergehende Abschnitte, in welche die Anwendung sich inhaltlich unterteilen lässt. Eine genauere Verteilung der Komponenten und deren Verantwortungsbereiche erfolgt in Kapitel 4.

3.1. JSONPath-Funktionsumfang

Um die in Sekundärziel S1 definierte vollständige Umsetzung der JSONPath-Funktionalitäten nach RFC-9535 [8] zu ermöglichen, müssen zunächst alle im RFC aufgeführten Funktionalitäten betrachtet werden. Diese lassen sich übersichtlich anhand der jeweils zugehörigen Komponenten von JSONPath-Queries aufführen.

3.1.1. Allgemein

Eine JSONPath-Abfrage oder -Query ist ein Ausdruck, der aus einem gegebenen JSON-Objekt eine Anzahl an Knoten auswählt und diese als Knotenliste ausgibt. Hierbei sind sowohl leere Listen als auch Listen mit nur einem Knoten mögliche Ergebnisse.

Eine JSONPath-Query setzt sich aus dem Wurzel-Identifizier und beliebig vielen Segmenten zusammen. Für die Auswertung eines JSONPath-Ausdrucks kann der Ausdruck ausgehend vom Wurzel-Identifizier segmentweise abgearbeitet werden, wobei jedes Segment als Rückgabewert eine Knotenliste liefert, die als Eingangswert des darauf folgenden Segments verwendet wird.

3.1.2. Wurzel

Der Wurzel-Identifizier ist der Ausgangspunkt für den im JSONPath-Ausdruck beschriebenen Pfad. JSONPath-Ausdrücke müssen immer an der Wurzel starten und können davon ausgehend durch die Baumstruktur des JSON-Objekts navigieren.

1 §

Der allein stehende Wurzel-Identifizier ist der kürzest mögliche JSONPath-Ausdruck und hat als Rückgabe eine Knotenliste mit dem Wurzelknoten des JSON-Objekts.

3.1.3. Segmente

Segmente können entweder Kinder- oder Nachfahrensegmente sein. Kindersegmente betrachten die direkten Kindknoten des aktuellen JSON-Wertes und geben null oder mehr dieser Kindknoten in der Rückgabeliste zurück. Nachfahrensegmente betrachten zusätzlich zu den Kindknoten des aktuellen Wertes auch alle weiteren Nachfahren der Knoten.

JSONPath-Ausdrücke können eine beliebige Kombination aus Kinder- und Nachfahrensegmenten beinhalten.

```
1  $['a']  
2  $.['a']
```

Der erste der obigen Ausdrücke liefert einen Kindknoten des Wurzelknotens mit Namen *a*, sofern dieser vorhanden ist. Der zweite Ausdruck liefert einen Nachfahren des Wurzelknotens mit Namen *a* zurück, sofern mindestens einer hiervon vorhanden ist.

Segmente können in Klammernotation oder Punktnotation geschrieben werden und die Notation kann innerhalb eines JSONPath-Ausdrucks beliebig wechseln. Inhaltlich führt die Notation zu keiner Änderung an der Auswertung des jeweiligen Segments.

Die Punktnotation ist eine vereinfachte Schreibweise, die einen eingeschränkteren Funktionsumfang aufweist. So können Namen und Wildcards sowohl in Klammernotation als auch in Punktnotation verwendet werden, während Indizes, Array-Slices und Filter nur in der Klammernotation verwendet werden können.

```
1  $['a']  
2  $.a  
3  $.['a']  
4  $.a
```

Die Punktnotation ist außerdem insofern eingeschränkt, dass jeweils nur ein Selektor pro Segment verwendet werden kann, während die Klammernotation einen oder mehr Selektoren pro Segment unterstützt.

```
1  $['a', 'b', 'c']
```

Das obige Beispiel zeigt ein Segment in Klammernotation, welches die Kindknoten mit Namen *a*, *b* und *c* des Wurzelknotens auswählt, sofern diese vorhanden sind. Eine solche Mehrfachauswahl ist in der vereinfachten Punktnotation nicht möglich.

3.1.4. Selektoren

Selektoren werden innerhalb von Segmenten verwendet und wählen null oder mehr Kindknoten des Eingangswertes aus, die sie zurückgeben. Konkret werden für JSONPath-Ausdrücke fünf Selektoren definiert:

- Namensselektoren
- der Wildcard-Selektor
- Slice-Selektoren
- Index-Selektoren
- Filterselektoren

Namensselektoren

Namensselektoren wählen maximal einen Kindknoten anhand seines Namens aus.

-
- | | |
|---|-------------|
| 1 | \$[' a '] |
| 2 | \$[" a "] |
-

Für die Verwendung innerhalb von Segmenten in Klammernotation können Namensselektoren mit einfachen oder doppelten Anführungszeichen als Strings angegeben werden. Diese Strings können die meisten Unicode-Zeichen beinhalten und über Escape-Sequenzen auch Sonderzeichen und Hex-Zeichen abbilden.

-
- | | |
|---|----------|
| 1 | \$. a |
| 2 | \$. . a |
-

Segmente in Punktnotation können nur eine vereinfachte Form von Namensselektoren verwenden, welche einen eingeschränkten Zeichensatz ermöglicht. Konkret sind hier außer dem Alphabet in Groß- und Kleinschreibung, Unterstrichen und Zahlen keine der ersten 128 Unicode-Zeichen erlaubt und es werden keine Escape-Sequenzen unterstützt. Außerdem darf die vereinfachte Form Zahlen nur ab dem zweiten Zeichen beinhalten.

Der Wildcard-Selektor

-
- | | |
|---|-------|
| 1 | \$[*] |
| 2 | \$.* |
-

Der Wildcard-Selector wird durch ein Multiplikationszeichen dargestellt und wählt alle Kindknoten des aktuellen Knotens aus.

Slice-Selektoren

-
- | | |
|---|-----------|
| 1 | \$[1:5:2] |
|---|-----------|
-

Slice-Selektoren wählen eine Teilmenge der Elemente eines Arrays anhand eines Anfangs- und Endindexes sowie einer Schrittgröße aus. Alle dieser Werte haben Standardwerte und müssen dementsprechend nur angegeben werden, wenn der Slice von diesen abweichen soll. Ein Slice wählt keine Elemente aus, wenn der Eingangswert kein Array ist.

Der Anfangsindex stellt die einschließende Anfangsgrenze der Ergebnismenge dar, während der Endindex die ausschließende Endgrenze des Ergebnisses darstellt. Werden keine Werte angegeben, wird das komplette Eingangsarray betrachtet.

Die Schrittgröße gibt an, wieviele der Elemente übernommen werden sollen und, über ihr Vorzeichen, in welcher Richtung das Array durchlaufen wird. Standardmäßig ist die Schrittgröße 1, sodass jedes Element in den definierten Grenzen in der Ergebnismenge landet.

Eine Schrittgröße von 2 würde zum Beispiel nur jedes zweite Element in die Ergebnismenge übernehmen und eine negative Schrittgröße von beispielsweise -1 würde das Array rückwärts entsprechend dem Betrag der Schrittgröße durchlaufen.

Im obigen Beispiel würde der Slice-Selektor von dem Element an Stelle 1 bis zu dem Element an Stelle 5 das Array mit einer Schrittgröße von 2 durchlaufen. Aus einem ausreichend langen Array würden also die Elemente an Stelle 1 und 3 ausgewählt.

Index-Selektoren

Index-Selektoren wählen aus einem Array einen Wert anhand seiner Position im Array aus. Es sind sowohl positive als auch negative Indizes erlaubt, wobei positive Indizes die Position im Array beginnend bei 0 vom Anfang des Arrays zählen und negative Indizes beginnend bei -1 vom Ende des Arrays zählen.

Indizes, die außerhalb des Arrays liegen, führen zu einer leeren Rückgabeliste. Ein Index-Selektor wählt kein Element aus, wenn der Eingangswert kein Array ist.

```

1  $[0]
2  $[-1]

```

Die obigen Beispiele wählen jeweils das erste beziehungsweise das letzte Element aus, sofern die Wurzel des JSON-Objekts ein Array aus mindestens einem Element ist.

Filterselektoren

Ein Filterselektor definiert einen logischen Ausdruck, anhand dessen die Knoten auf der aktuellen Objektebene oder die Elemente eines Arrays gefiltert werden. Dieser logische Ausdruck, auch Filterausdruck genannt, wird für jeden betrachteten Knoten beziehungsweise jedes betrachtete Element evaluiert und entscheidet, ob dieses ausgewählt wird.

Während der Auswertung kann der Ausdruck auf den aktuell betrachteten Knoten beziehungsweise das aktuell betrachtete Element als *aktuellen Knoten (current node)* zugreifen. Dieser aktuelle Knoten kann als Ausgangspunkt für eine oder mehrere JSONPath-Abfragen für Teilausdrücke des Filterausdrucks verwendet werden.

Jede JSONPath-Abfrage kann als Existenztest, zum Auslesen eines spezifischen Wertes oder als Funktionsargument verwendet werden.

```

1  $[?@.a == 1]

```

Das obige Beispiel zeigt einen Filterausdruck, welcher aus einem Vergleichsausdruck zwischen einem potenziellen Kindknoten des aktuellen Knotens und der Zahl 1 besteht.

Filterausdrücke bestehen auf oberster Ebene zunächst aus logischen Ausdrücken, welche entsprechend der booleschen Algebra dis- und konjugiert sowie negiert und geklammert werden können. Grundelement dieser Ausdrücke bilden Vergleichs- und Testausdrücke.

Ein Testausdruck überprüft entweder, ob ein Knoten durch eine eingebettete Abfrage gefunden wird, oder ob das Ergebnis eines Funktionsausdrucks entsprechend des Rückgabetyps der Funktion ein logisches *wahr* oder eine nicht leere Knotenliste ist. Funktionen, die nicht entweder einen logischen Wert oder eine Knotenliste zurückgeben, dürfen nicht in Testausdrücken verwendet werden.

Vergleichsausdrücke ermöglichen Vergleiche zwischen primitiven Werten der Typen *Zahl*, *String*, *true*, *false* und *null*. Diese können als Literale angegeben werden, das Ergebnis einer singulären Abfrage sein oder als Ergebnis eines Funktionsaufrufs mit entsprechendem Rückgabewert übergeben werden.

Funktionsausdrücke

Funktionsausdrücke können je nach Rückgabebetyp entweder in Testausdrücken oder als Teil eines Vergleichsausdrucks verwendet werden. Spezifisch wird von JSONPath die Möglichkeit für Funktionserweiterungen über einen Erweiterungspunkt gestellt, um die Funktionalität von Filterausdrücken zu erweitern.

Eine Funktionserweiterung muss einen eindeutigen Namen definieren, welcher auf null oder mehr Argumente angewandt ein Ergebnis liefert. Funktionen müssen in ihrer Auswertung frei von Nebeneffekten definiert werden, sodass die Ausführungsreihenfolge von Ausdrücken und ein möglicher vorzeitiger Abbruch der Ausführung keinen Unterschied im Ergebnis für den Ausdruck ergeben.

Funktionen müssen in Ausdrücken wohlgeformt und wohltypisiert verwendet werden und die Einhaltung dieser Konventionen muss unabhängig vom betrachteten JSON-Objekt sichergestellt werden.

Rückgabewerte von Funktionen können drei verschiedene Typen annehmen: Wert, Boolean und Knoten. Rückgaben vom *Wert*-Typ sind entweder JSON-Werte oder *Nothing*. *Boolesche* Rückgaben können entweder ein logisches *Wahr* oder ein logisches *Falsch* sein. Rückgaben vom Typ *Knoten* müssen eine Knotenliste zurückgeben, die jedoch auch leer sein kann.

Der Rückgabewert *Nothing* ist ein gesondertes Ergebnis, welches die Abwesenheit eines JSON-Wertes repräsentiert und sich von allen JSON-Werten - inklusive *null* - unterscheidet. Ebenso sind das logische *Wahr* und das logische *Falsch* als Rückgabe vom Typ Boolean und die in JSON-Werten verwendeten Literale *true* und *false* voneinander zu trennen.

Neben der Möglichkeit, über den Erweiterungspunkt eigene Funktionen zu definieren, stellt JSONPath auch ein paar Standardfunktionen als Ausgangspunkt zur Verfügung:

- `length()`
- `count()`
- `match()`
- `search()`
- `value()`

3.1.5. Whitespaces

JSONPath erlaubt an vielen Stellen beliebige Benutzung der Whitespaces Leerzeichen, Tabulator, Zeilenumbruch und Wagenrücklauf. Whitespaces tragen nicht zum Informationsgehalt der Ausdrücke bei und können nur zur übersichtlicheren Formatierung dieser verwendet werden.

In den hier aufgeführten Beispielen werden zur Vereinfachung nur Leerzeichen betrachtet; nahezu jede Stelle, an der Leerzeichen in JSONPath-Queries erlaubt sind, erlaubt auch die anderen bereits aufgeführten Whitespace-Zeichen.

Beispielsweise erlaubt sind Whitespaces zwischen Segmenten und Selektoren innerhalb von geklammerten Segmenten. So sind zum Beispiel beide folgende Ausdrücke gleichermaßen erlaubt und inhaltlich identisch.

```
1  $['a']['b',1].c
2  $ [ 'a' ] [ 'b' , 1 ] .c
```

Da die Whitespaces in JSONPath explizit deklariert werden, sind neben den Stellen, an denen sie beliebig erlaubt sind, auch solche Stellen erkennbar, an denen kein Whitespace stehen darf. Eine RFC-konforme Implementierung muss also an diesen Stellen darauf achten, dass die Eingabe keine Whitespaces aufweist.

Konkret sind die Stellen, an denen die Eingabe keine Whitespaces enthalten darf:

- vor dem Wurzel-Identifizier und nach dem letzten Segment
- zwischen Punkt- beziehungsweise Nachfahren-Operator und Selektoren
- innerhalb von geschlossenen Definitionen, einschließlich Strings
- zwischen Funktionsnamen und -klammern
- innerhalb der Klammern von Singular Queries

Whitespaces vor und nach der JSONPath-Abfrage

Whitespaces vor beziehungsweise nach der JSONPath-Abfrage können potenziell ignoriert werden, indem diese einfach nicht als Teil des Ausdrucks betrachtet und bei Bedarf vor der Verarbeitung entfernt werden.

Diese Verwendung lässt sich auch mit der Definition im RFC vereinen, da dieser sich nur auf die Ausdrücke selbst und nicht deren eventuell sie umgebende Whitespaces eingeht. Es obliegt also einer gewissen freien Interpretation, ob diese Whitespaces explizit zu beachten sind oder nicht.

Whitespaces in geschlossenen Definitionen

Unter geschlossenen Definitionen sind in diesem Kontext meist kleinere Definitionen zu verstehen, die einen atomaren Baustein des JSONPath-Ausdrucks beschreiben. Beispiele hierfür sind Strings, Zahlen, Namen und Operatoren, insbesondere solche, die aus mehreren Zeichen bestehen. Diese Bausteine dürfen nicht durch Whitespaces aufgetrennt werden, auch wenn es sich hierbei oft mehr um eine Stilkonvention und weniger um technische Notwendigkeit handelt.

Obgleich in Strings keine Whitespaces im allgemeinen Sinne erlaubt sind, können Leerzeichen in Strings weiterhin beliebig verwendet werden. Diese Leerzeichen sind jedoch von inhaltlicher Relevanz, so sind beispielsweise die folgenden beiden Ausdrücke nicht inhaltlich deckungsgleich und verweisen auf unterschiedlich benannte Kindknoten des Wurzelknotens.

1	<code>\$['a']</code>
2	<code>\$['a ']</code>

In Strings dürfen keine tatsächlichen Tabulatoren, Zeilenumbrüche und Wagenrückläufe verwendet werden. Um diese darzustellen, können die Escape-Sequenzen `\t`, `\n` und `\c` verwendet werden.

Whitespaces in singulären Abfragen

Singuläre Abfragen ähneln in ihrer Definition stark allgemeinen JSONPath-Abfragen, jedoch dürfen sie anders als allgemeine Abfragen keine Whitespaces innerhalb ihrer geklammerten Segmente enthalten. Dies führt zu interessanten Randfällen, wie zum Beispiel:

1	<code>\$[?@['a']]</code>
2	<code>\$[?@['a']==1]</code>

Im oberen Ausdruck ist die innere Unterabfrage `$[ 'a' ]` Teil eines Testausdrucks, bei welchem auf die Existenz eines Kindknotens mit Namen `a` unter dem aktuellen Knoten geprüft wird. Dadurch handelt es sich bei der inneren Abfrage um eine relative Filterabfrage, innerhalb von welcher unter anderem die im Beispiel verwendeten Leerzeichen erlaubt sind.

Der untere Ausdruck hingegen enthält einen Vergleichsausdruck und betrachtet den Wert von einem eventuellen Kindknoten des aktuellen Knotens mit Namen `a`, den er mit dem Wert `1` vergleicht. `$[ 'a' ]` muss also eine singuläre Abfrage sein und erlaubt folglich keine Whitespaces innerhalb ihrer Klammern.

3.1.6. Normalisierte JSONPath-Ausdrücke

1	<code>\$. a</code>
2	<code>\$ [' a ']</code>
3	<code>\$ [" a "]</code>

Die Knoten eines JSON-Objekts können, wie im Beispiel oben illustriert, oft durch verschiedene JSONPath-Ausdrücke beschrieben werden. Um eine eindeutige Beschreibung für Knoten zu ermöglichen, mithilfe welcher zum Beispiel einfacher doppelte Einträge aus Knotenlisten gefiltert werden können, definiert RFC-9535 normalisierte JSONPath-Ausdrücke.

Normalisierte JSONPath-Ausdrücke sind JSONPath-Ausdrücke mit eingeschränkter Funktionalität, sodass diese, wenn sie auf das zugehörige JSON-Objekt angewendet werden, genau den einen Knoten zurückliefern, auf den sie verweisen. So existiert für jeden Knoten in einem JSON-Objekt genau ein normalisierter JSONPath-Ausdruck, der auf diesen Knoten verweist.

Konkret verwenden normalisierte JSONPath-Ausdrücke nur Namen- und Indexsegmente in der Klammernotation und verwenden einfache Anführungszeichen für die Namen. Ein Segment in einem *normalized path* kann außerdem nur jeweils einen Selektor beinhalten und es können nicht wie in allgemeinen Ausdrücken Whitespaces verwendet werden.

Um eine eindeutige Schreibweise für Namensselektoren zu ermöglichen, wird außerdem definiert, welche Zeichen mit einer Escapesequenz darzustellen sind und welche Escapesequenz genau zu verwenden ist. Alle anderen Zeichen sind ohne Escapesequenz darzustellen.

Indizes können nicht wie in allgemeinen JSONPath-Ausdrücken negativ sein, um das betrachtete Array von hinten zu traversieren, sondern müssen mit positiven Werten die Position des Elements vom Anfang des Arrays aus betrachtet angeben.

Hex-Character-Escapes können in normalisierten Ausdrücken nur Zahlen und Kleinbuchstaben enthalten und nicht auch Großbuchstaben, wie dies in allgemeinen JSONPath-Ausdrücken der Fall ist.

3.1.7. Erlaubte Eingangszeichen

Obgleich alle für JSONPath benötigten Sonderzeichen im Bereich der 128 7-Bit-ASCII-Zeichen liegen, erlauben JSONPath-Ausdrücke in den meisten Stellen, die eine Form von Strings beinhalten, die Verwendung von *Unicode*-Zeichen. Dies bedeutet, dass für die vollständige Erkennung und Verarbeitung aller möglichen JSONPath-Ausdrücke die Verarbeitung von Unicodezeichen notwendig ist.

Eine Reduktion des Zeichenraums auf die 7-Bit-ASCII-Zeichen schränkt die Verarbeitung von JSONPath-Ausdrücken jedoch nicht in ihrem Funktionsumfang ein, sondern reduziert lediglich den verfügbaren Namensraum für Strings innerhalb des Ausdrucks und damit indirekt auch innerhalb des zugrundeliegenden JSON-Objekts.

Beachtung von Groß- und Kleinschreibung

Für die Verarbeitung von JSONPath-Abfragen müssen Groß- und Kleinbuchstaben unterschieden werden können (die Grammatik ist *case sensitive*).

Wie die der Auswertung eines JSONPath-Ausdrucks zugrundeliegende JSON Struktur müssen auch Verweise darauf im Ausdruck - zum Beispiel in Form von Schlüsseln oder Strings - dieselbe Groß- beziehungsweise Kleinschreibung enthalten, um erfolgreich zugeordnet werden zu können.

Darüber hinaus gibt es auch für die Überprüfung von JSONPath-Ausdrücken auf deren Wohlgeformtheit und Validität Stellen, an denen Groß- und Kleinschreibung relevant sind. Die Schlüsselwörter *true*, *false* und *null* in logischen Ausdrücken müssen beispielsweise immer in Kleinbuchstaben geschrieben werden und Funktionsnamen dürfen keine Großbuchstaben enthalten.

Es gibt außerdem mehrere Stellen, in welchen die im RFC angegebene ABNF explizite Unicode-Codes für einzelne Buchstaben angibt, wodurch diese nur in der gegebenen Groß- beziehungsweise Kleinschreibung erlaubt sind. Dies betrifft beispielsweise die meisten Escape-Sequenzen, welche nur für die jeweiligen Kleinbuchstaben definiert sind, und normalisierte Hex-Character, in welchen nur Kleinbuchstaben erlaubt sind.

Die Angabe eines Exponenten bei Zahlen in logischen Ausdrücken beispielsweise erlaubt hingegen sowohl ein großes als auch ein kleines *E*.

3.1.8. Fehlerbehandlung

RFC-9535 definiert für JSONPath-Implementierungen, dass diese Fehlermeldungen für nicht wohlgeformte und valide JSONPath-Ausdrücke ausgeben müssen. Wohlgeformtheit und Validität eines JSONPath-Ausdrucks lassen sich unabhängig von den JSON-Objekten bestimmen, auf die der Ausdruck angewendet werden soll.

Ein JSONPath-Ausdruck ist wohlgeformt, wenn er der im RFC definierten und in ABNF notierten Grammatik entspricht. Des Weiteren ist ein wohlgeformter JSONPath-Ausdruck valide, wenn die in ihm vorkommenden Integer-Werte, die für die Verarbeitung des Ausdrucks verwendet werden - Indizes und Schrittgrößen - im Bereich der in RFC-7493 definierten Integer-Werte liegen und im Ausdruck verwendete Funktionen wohl-typisiert sind.

JSONPath-Implementierungen dürfen nicht ohne Angabe eines Fehlers fehlschlagen. Ein JSONPath-Ausdruck, der von einer anderen Struktur ausgeht als das JSON-Objekt, auf das er angewendet wird, kann zu einer leeren Ergebnismenge führen, wobei es sich nicht um einen Fehler der Implementierung handeln muss.

3.2. Aufbau des Compilers

Um den Compiler im Sinne von Sekundärziel S6 möglichst einfach verständlich und wartbar zu gestalten, empfiehlt sich die Verwendung von einer Unterteilung anhand der in Kapitel 2 beschriebenen Phasen von Compilern. Da eine gezielte Fehlerbehandlung über den Umfang dieser Arbeit hinausgeht, sind für den Compiler spezifisch die Phasen der *lexikalischen Analyse*, *syntaktischen Analyse* und des *Zwischencodes* relevant.

3.2.1. Lexikalische Analyse

Die lexikalische Analyse eines JSONPath-Ausdrucks erfordert zunächst eine präzise Verarbeitung der Eingangszeichen entsprechend der in RFC-9535 definierten ABNF. Es muss sichergestellt werden, dass nur erlaubte Zeichen akzeptiert werden und als entsprechende Token für die weitere Verarbeitung aufbereitet werden.

Die Definition der verwendeten Tokens muss hierbei in Absprache mit der syntaktischen Analyse erstellt werden, sodass sowohl die Möglichkeiten der lexikalischen Analyse als auch die Anforderungen der syntaktischen Analyse beachtet werden.

Wie bereits beschrieben, erfordert JSONPath nach RFC-9535 die Verarbeitung von Unicodezeichen, kann jedoch auch mit eingeschränkten Namensräumen durch kleinere Zeichensätze wie 7-Bit-ASCII realisiert werden. Dementsprechend muss 7-Bit-ASCII als Mindestmaß von der lexikalischen Analyse verarbeitet werden können und eine Lösung für den größeren Unicodezeichensatz gefunden werden.

3.2.2. Syntaktische Analyse

Die syntaktische Analyse eines JSONPath-Ausdrucks muss überprüfen, ob der Ausdruck aus der JSONPath-Grammatik hergeleitet werden kann oder nicht. Hierfür muss zunächst definiert werden, welche Prüfungen in der syntaktischen Analyse abgehandelt werden können, welche auf die vorausgehende lexikalische Analyse ausgelagert werden können und welche in einem weiteren Verarbeitungsschritt umgesetzt werden müssen.

Spezifisch muss entschieden werden, welche Regeln der Grammatik wie repräsentiert werden können. Für Regeln, die eine Anpassung erfordern, muss eine geeignete solche gefunden werden, ohne dadurch die akzeptierten und nicht-akzeptierten Ausdrücke der Grammatik zu beeinflussen.

3.2.3. Zwischencode

Das Ergebnis des Compilers beziehungsweise die Schnittstelle zum Interpreter und der weiteren Verarbeitung und Auswertung der Eingabe stellt der Zwischencode dar. Konkret soll hierfür ein Syntaxbaum angestrebt werden, welcher die zur Ausführung benötigten Operationen repräsentiert.

Der Syntaxbaum kann vor der Auswertung durch den Interpreter von einem separaten Modul vereinfacht und möglicherweise optimiert werden. Implementationsdetails von JSONPath, die auf die Auswertung keinen direkten Einfluss haben, können aus dem Syntaxbaum entfernt werden, um den Syntaxbaum zu einem abstrakteren Syntaxbaum umzuformen.

Dadurch kann die Kopplung zwischen Compiler und Interpreter weiter gelockert werden und durch die erhöhte Modularität die Wartbarkeit und Erweiterbarkeit verbessert werden.

3.3. Aufbau des Interpreters

Der Interpreter hat die Aufgabe, den als Zwischencode verwendeten Syntaxbaum traversierend die Auswertung des Eingangsausdrucks anhand eines Datensatzes in YottaDB umzusetzen. Hierfür muss dieser auf geeignete Weise die YottaDB-Instanz ansprechen und Daten abfragen können, um diese Daten intern auszuwerten.

Die Kompetenzverteilung zwischen YottaDB und dem Interpreter ist an dieser Stelle relevant. Für die Auswertung benötigt der Interpreter stellenweise größere Datenmengen aus dem Datensatz, sodass im Extremfall ein möglicher Ansatz wäre, vor Beginn der Auswertung den kompletten Datensatz in den Interpreter zu laden und dort zu auswerten.

Daraus würde aber insbesondere für große Datensätze ein Anspruch von effizienter Datenverarbeitung an den Interpreter entstehen - vorausgesetzt, der Datensatz kann überhaupt gänzlich in den Interpreter geladen werden und übersteigt nicht dessen beispielsweise aus der verfügbaren Hardware entstehende technische Limitationen.

Ebenso unpraktisch ist der andere Extremansatz, die Daten zu jeder Zeit vollständig in YottaDB zu lassen und nur über gezielte Abfragen auszuwerten, da diese Abfragen einen Flaschenhals für die Umsetzung der JSONPath-Funktionalität darstellen können und folglich die Auswertung ineffizient werden kann.

Um ein besseres Bild zu erhalten, welche JSONPath-Funktionalitäten wie realisiert werden können, kann eine Art Mapping zwischen JSONPath und YottaDB erstellt werden, anhand welchem sich die Implementierung des Interpreters orientieren kann.

3.4. Beschleunigung der Suche in YottaDB

Um die Suche in großen Datensätzen in YottaDB beschleunigen zu können, müssen mögliche Lösungsansätze zunächst theoretisch betrachtet werden, um ihre prinzipielle Eignung zu überprüfen. Hierfür muss zunächst die Ausgangssituation genauer betrachtet werden.

Prinzipiell kann zwischen zwei Arten der Suche unterschieden werden, die durch die Auswertung eines JSONPath-Ausdrucks auftreten können: die Suche nach einem Eintrag mit exakter Übereinstimmung, wie dies beispielsweise bei Namensselektoren der Fall ist, und die Suche nach einer Übereinstimmung zwischen einem Teil eines Eintrags und einem regulären Ausdruck, wie dies zum Beispiel in den in JSONPath-Ausdrücken verwendbaren Funktionen *search* und *match* auftritt.

Da die Suche nach exakten Übereinstimmungen der für JSONPath-Ausdrücke kritischere und häufiger auftretende Fall ist, werden im Weiteren Ansätze zu deren möglicher Beschleunigung auf großen Datensätzen genauer betrachtet. Die Beschleunigung der Suche nach Übereinstimmungen zwischen einem Teil des Eintrags und einem regulären Ausdruck wird hier nicht weiter verfolgt.

3.4.1. Laufzeit von Suchen nach exakter Übereinstimmung

Eine einfache lineare Suche in einem Datensatz aus n Einträgen hat eine Laufzeit von $\mathcal{O}(n)$, da jeder Eintrag nacheinander betrachtet wird und im schlimmsten Fall alle n Einträge durchsucht werden müssen, bevor der gesuchte Eintrag gefunden wird.

Da YottaDB sich selbst als *die schnellste Schlüssel-Wert-Datenbank der Welt* bezeichnet [13] und Schlüssel in YottaDB immer sortiert behandelt werden, kann davon ausgegangen werden, dass YottaDB binäre Suche oder einen Algorithmus mit ähnlicher Laufzeit verwendet und somit eine Laufzeit von $\mathcal{O}(\log(n))$ erreicht. Solange YottaDB als eine *Black Box* behandelt wird, können ohne genauere Angaben oder konkrete Laufzeitmessungen keine präziseren Annahmen aufgestellt werden.

Konkret bedeutet das, dass ein Algorithmus, der die Suche in YottaDB beschleunigen soll, die Laufzeit von $\mathcal{O}(\log(n))$ weiter reduzieren muss, um für weitere Betrachtungen relevant zu sein. Im Allgemeinen kann ein Laufzeitgewinn durch erhöhten Speicherbedarf *erkauft* werden, weshalb auch der Speicherbedarf eine relevante Größe für die Bewertung von Algorithuskandidaten darstellt.

3.4.2. Speicherkomplexität von Suchen nach exakter Übereinstimmung

Für den Speicherbedarf ist neben der Anzahl n der zu durchsuchenden Einträge auch die Länge m dieser Einträge relevant. Da die Einträge unterschiedliche Längen aufweisen können, wird in dieser Betrachtung davon ausgegangen, dass m die Maximallänge und die Länge aller Einträge darstellt. Einträge mit einer Länge, die kleiner als m ist, können als effizienter betrachtet werden, da diese andernfalls einfach auf Länge m aufgefüllt werden könnten.

Unter der Annahme, dass YottaDB die Einträge ohne nennenswerten Überhang abspeichern kann, ist die Speicherkomplexität in der Ausgangssituation $n * m$, da n Einträge der Länge m gespeichert werden müssen. Wie auch bei der Laufzeit können präzisere Angaben ohne konkrete Speichermessungen nicht getroffen werden, jedoch ist die tatsächliche genaue Speichergröße in der Ausgangssituation nicht relevant, da die Speicherkomplexität lediglich als zusätzliches Entscheidungskriterium für die Auswahl eines Algorithmus verwendet wird.

Die eventuelle Optimierung der Speicherkomplexität ist nicht das Ziel der Beschleunigung der Suche, jedoch sollte die Speicherkomplexität nicht komplett ausarten, da die Beschleunigung immer noch realistisch umsetzbar bleiben muss.

3.4.3. Parametrisierung der Suchalgorithmen

Sofern es sich für den jeweiligen Ansatz anbietet, kann dieser über geeignete Parametrisierung sinnvoll gesteuert und an die tatsächlichen Umstände des Datensatzes besser angepasst werden. Dadurch kann sich das Einsatzgebiet des jeweiligen Ansatzes über dessen grundlegende Annahmen hinaus etwas erweitern lassen.

Hierbei müssen für jeden Parameter dessen Auswirkungen auf Speicherkomplexität und Laufzeit abhängig ergründet werden und sofern möglich müssen Geltungsbereiche und sinnvolle Standardwerte für jeden Parameter definiert werden.

3.4.4. Zusammenfassung

Zusammenfassend lässt sich also als Auswahlkriterium für betrachtete Algorithmen zur Beschleunigung der Suche in YottaDB festhalten, dass diese eine Laufzeitverbesserung gegenüber von $\mathcal{O}(\log(n))$ erbringen müssen und gleichzeitig nicht unrealistisch weit mehr Speicherkomplexität als $n * m$ mit sich bringen dürfen.

Nach der Implementierung eines oder mehrerer geeigneter Algorithmen können durch praktische Messungen genauere Vergleiche zwischen den verschiedenen Ansätzen in diversen Szenarien gezogen werden.

3.5. Anforderungsverzeichnis

Im Folgenden werden die in diesem Kapitel beschriebenen Aspekte in einer Anforderungsliste zusammengefasst. Hierbei handelt es sich bewusst nicht nur um Anforderungen, die im Rahmen dieser Arbeit abgehandelt werden können, sondern auch Anforderungen für eine mögliche zukünftige Weiterentwicklung des Systems.

1. JSONPath-Funktionsumfang

- a) Wurzel- und *Current*-Identifizier
- b) Segmente in Klammer- und Punktnotation
- c) Namensselektoren als Strings und Member-Name-Shorthands
- d) Wildcard-Selektor
- e) Slice-Selektoren mit optionalen Parametern und Standardwerten
- f) Index-Selektoren für positive und negative Indizes
- g) Filterselektoren mit logischen Ausdrücken
 - i. Testausdrücke
 - ii. Vergleichsausdrücke
 - iii. Funktionsausdrücke mit Funktionserweiterungen
- h) Beachtung von Whitespaces
- i) Beachtung von Groß- und Kleinschreibung
- j) Normalisierte JSONPath-Ausdrücke
- k) Fehlerbehandlung

2. JSONPath-Compiler

- a) Kompetenzverteilung zwischen lexikalischer Analyse, syntaktischer Analyse und Nachbearbeitung
- b) Lexikalische Analyse
 - i. Präzise Verarbeitung der Eingangszeichen
 - ii. Tokenisierung passend zur Grammatik
 - iii. Abbildung von 7-Bit-ASCII (Minimum)

- iv. Abbildung von Unicode (RFC-konform)
- c) Syntaktische Analyse
 - i. Überprüfung der Wohlgeformtheit der Eingabe
 - ii. Umsetzung der Grammatik
 - iii. Umformungen der Grammatikregeln unter Einhaltung von RFC-Konformität
- d) Zwischencode
 - i. Überführung in Syntaxbaum
 - ii. Abbildung der Auswertungsoperationen und deren Parameter
- e) Nachbearbeitung
 - i. Überprüfung der Validität der Eingabe
- 3. Zwischencodeoptimierung
- 4. JSONPath-Interpreter
 - a) Auswertung des Zwischencode-Syntaxbaums
 - b) Schnittstelle zu YottaDB
 - c) Kompetenzverteilung zwischen YottaDB und Interpreter
 - d) Mapping von JSONPath-Funktionalitäten auf YottaDB-Abfragen
- 5. Beschleunigen der Suche in YottaDB
 - a) Auswahl geeigneter Algorithmen
 - b) Theoretische Bewertung anhand Laufzeit und Speicherkomplexität
 - c) Parametrisierung der Algorithmen
 - i. Bewertung der jeweiligen Einflüsse auf den Algorithmus
 - ii. Definition von Geltungsbereichen und Standardwerten
 - d) Implementation eines oder mehrerer Algorithmen
 - e) Vergleich verschiedener Ansätze anhand von Laufzeit- und Speichermessungen

4. Lösungskonzept

Anhand der in Kapitel 3 erarbeiteten Anforderungen wird im Folgenden ein Lösungskonzept erstellt. Hierbei werden sowohl zielführende Überlegungen als auch exemplarisch einige nicht zielführende Ansätze erkundet, um einen Überblick über die getroffenen Entscheidungen und die dazu betrachteten Alternativen zu geben.

4.1. Gesamtsystem

Das Gesamtsystem lässt sich zunächst inhaltlich anhand des Aufgabenbereichs in Compiler und Interpreter unterteilen, welche die beiden Hauptabschnitte des Programmablaufs darstellen. Um den Gesamtablauf zu steuern, Eingaben vom Nutzer entgegen zu nehmen und bei Bedarf die Schnittstelle zwischen einzelnen Komponenten zu spielen, werden diese beiden Hauptabschnitte in eine zentrale Steuerungskomponente eingebettet.

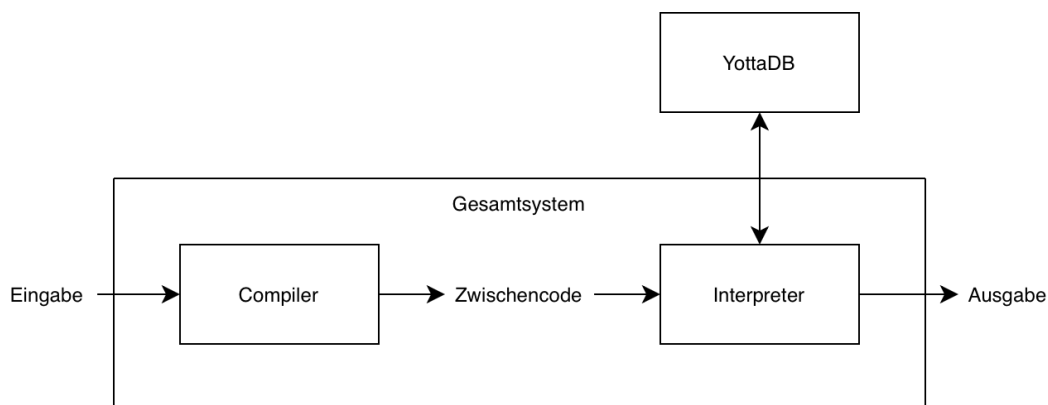


Abbildung 4.1.: Architekturübersicht des Gesamtsystems

Wie in Abbildung 4.1 erkennbar wird, kann das Gesamtsystem als eine Art Pipeline verstanden werden, welche eine Eingabe in Form eines JSONPath-Ausdrucks entgegennimmt und auswertet, um eine entsprechende Ausgabe zurückzugeben.

Der JSONPath-Ausdruck wird dabei zunächst vom Compiler in einen Syntaxbaum als Zwischencode überführt, und anschließend vom Interpreter unter Bezug einen Datensatz auszuwerten, der in einer YottaDB-Instanz gespeichert wird.

4.1.1. Systemarchitektur

Um eine präzisere Vorstellung der internen Abläufe des Zielsystems zu bekommen, wird ausgehend von der Architekturübersicht in Abbildung 4.1 ein detailliertes Architekturdiagramm erstellt. Hierbei werden die Hauptsegmente in einzelne Komponenten unterteilt und die Darstellung um genauere Ablaufschemata erweitert.

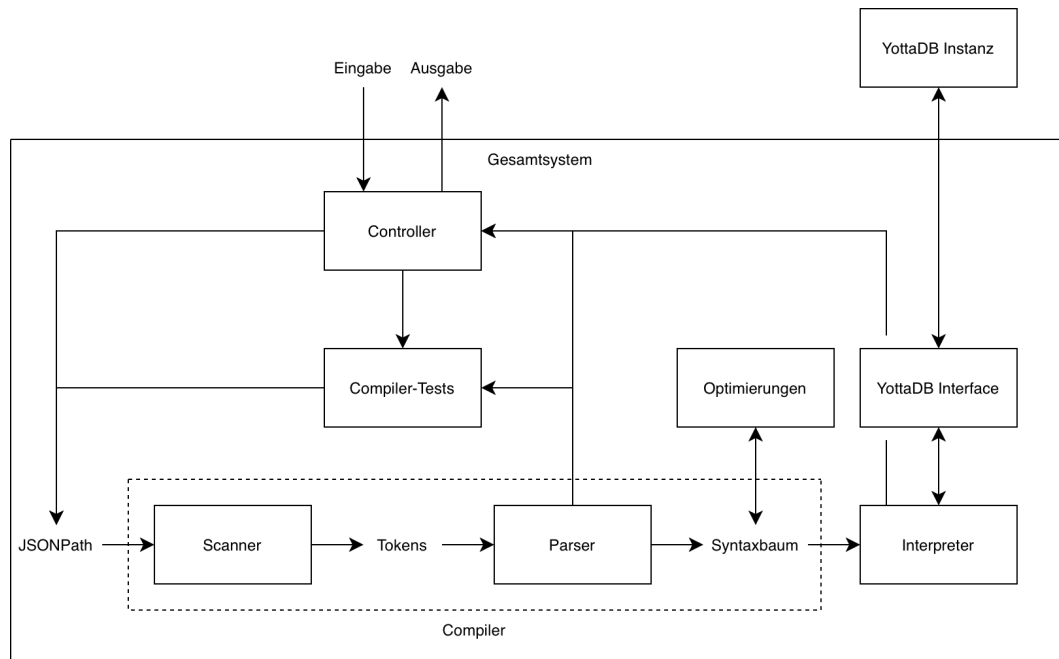


Abbildung 4.2.: Architekturdiagramm im Detail

Abbildung 4.2 zeigt das detaillierte Architekturdiagramm. Neben kleinen Anpassungen für eine ansprechendere Darstellung ist dies in seiner Funktionalität mit der Übersicht identisch, ermöglicht es jedoch, die einzelnen Komponenten und deren Verantwortungsbereiche besser nachzuvollziehen.

Die Eingabe wird zunächst vom *Controller* entgegengenommen, welcher die zentrale Steuereinheit des Systems darstellt. In der Eingabe können neben JSONPath-Ausdrücken auch diverse Steuersequenzen mitgegeben werden, um verschiedene Abläufe auszulösen.

Im Standardfall reicht der Controller den JSONPath-Ausdruck an den Scanner weiter, welcher diesen in Tokens zerlegt. Basierend auf diesen Tokens kann der Parser überprüfen, ob der Ausdruck aus der JSONPath-Grammatik hervorgehen kann. Das Ergebnis des Parsers ist zum einen die Information, ob der Ausdruck ein wohlgeformter und valider JSONPath-Ausdruck ist, und zum anderen im Falle, dass der Ausdruck akzeptiert wurde, der daraus entstehende Syntaxbaum.

Im Falle, dass der Ausdruck vom Parser verworfen wird, bricht der Controller die Auswertung direkt ab, andernfalls leitet er den Syntaxbaum an den Interpreter weiter. Der Interpreter greift dann zur Auswertung des Syntaxbaums über eine geeignete YottaDB-Schnittstelle auf den Datensatz in YottaDB zu. Das Ergebnis der Auswertung wird anschließend zurück zum Controller gesendet, sodass dieser die Ausgabe in geeigneter Form entsprechend des Ergebnisses tätigen kann.

Im Architekturdiagramm sind außerdem automatisierte Tests und Optimierungen vorgesehen. Beide werden bei Bedarf vom Controller aus angesteuert.

Die automatisierten Tests beziehen sich im Architekturdiagramm zunächst nur auf den Compiler und dessen Eingabevalidierung. Eine Erweiterung um Tests des Gesamtsystems, also Compiler und Parser, erfordert eine Möglichkeit, das YottaDB-Interface auf einen Testdatensatz umzulenken, da die Auswertung von JSONPath-Ausdrücken von dieser zugrundeliegenden Datensatz abhängig ist.

Die Compiler-Tests simulieren die Eingabe von diversen JSONPath-Ausdrücken und überprüfen das Ergebnis des Parsers. Idealerweise sollten diese Ausdrücke möglichst viele verschiedene Testfälle und Szenarien abdecken, sodass die Funktionalität des Parsers zum Beispiel nach eventuellen Anpassungen reibungslos und möglichst ausführlich überprüft werden kann.

Die Komponente für Optimierungen des Syntaxbaums kann diesen vor der Weiterverarbeitung durch den Interpreter vereinfachen. Ziel dieser Vereinfachung ist nicht, die Laufzeit des Interpreters zu beschleunigen, da die Laufzeitgewinne wahrscheinlich maximal die Laufzeit der Optimierung selbst aufwiegen können. Die Optimierung dient vielmehr der Erhöhung der Modularität zwischen Compiler und Interpreter und der Vereinfachung und dadurch Reduktion möglicher Fehlerquellen des Interpreters.

Zusätzlich kann an Stelle der Optimierung auch eine Nachbearbeitung des Syntaxbaums erfolgen, in welcher Prüfungen des Ausdrucks abgehandelt werden können, zu denen der Parser nicht in der Lage ist. Insbesondere sind das diese der Wohlgeformtheit und Wohltypisiertheit von Funktionsausdrücken und dementsprechend beispielsweise auch der Typisierung der logischen Ausdrücke in Filterausdrücken.

4.1.2. Kompetenzverteilung Scanner / Parser

Die in RFC-9535 definierte ABNF-Grammatik bricht JSONPath-Ausdrücke von ihrer Gesamtheit ebenenweise in kleinere Komponenten herunter, bis diese zeichenweise anhand von Unicode-Zeichen präzise definiert werden können. Eine Umsetzung dieser Grammatik mittels eines Scanners für die lexikalische Analyse und eines Parsers für die syntaktische Analyse muss also eine sinnvolle Aufteilung der ABNF-Regeln definieren, anhand welcher die Schnittstelle von Scanner und Parser vollzogen wird.

Wohl die einfachsten Lösungen für die Regelverteilung sind die beiden Extreme, nämlich die vollständige Umsetzung der Regeln in entweder Scanner oder Parser. Beide Fälle haben den Vorteil, dass die Regeln aus dem RFC mit minimalen Umformungen verwendbar sind, wodurch die Umformung als potenzielle Fehlerquelle eingeschränkt werden kann. Außerdem ist der jeweils unterbelastete Teil des Systems so einfach wie möglich in der jeweiligen Implementierung - so muss zum Beispiel der Scanner nur alle erlaubten Eingangszeichen unverändert an den Parser weiterleiten, wenn alle Regeln im Parser auf Zeichenebene behandelt werden sollen.

Im Falle der vollständigen Umsetzung der Regeln im Scanner, wie dies beispielsweise über reguläre Ausdrücke möglich wäre, können die Angaben des RFCs zu den erlaubten Zeichen sehr präzise realisiert werden. Jedoch kann dieser Scanner nur zum Annehmen beziehungsweise Ablehnen von Ausdrücken verwendet werden und erfordert intensivere Nachbearbeitung eines akzeptierten Ausdrucks, um diesen letztlich ausführen zu können.

Der gegenteilige Ansatz hierzu, die Regeln auf Zeichenebene im Parser zu realisieren, bietet prinzipiell auch die Möglichkeit, die einzelnen Zeichen sehr präzise zu verarbeiten und kann durch die Erzeugung eines Syntaxbaums die Weiterverarbeitung der Eingabe zusätzlich zur Verwerfung falscher Eingaben unterstützen.

Dieser Ansatz führt jedoch zu sehr vielen Regeln im Parser, wodurch dieser erheblich fehleranfälliger und schwerer zu warten wird. Weiterhin erhöht sich die Komplexität des entstehenden Syntaxbaums, da auch dieser auf Zeichenebene aufgestellt wird, sodass mehr Aufwand in einem folgenden Vereinfachungsschritt des Syntaxbaums nötig wird.

Es gilt also, einen sinnvollen Kompromiss zu finden, der die jeweiligen Vorzüge von Scanner und Parser möglichst gut ausnützt und die jeweiligen Schwächen wo möglich kompensiert.

Da die Weiterverarbeitung des Ausdrucks beziehungsweise seine Ausführung oder Auswertung ermöglicht werden soll, wird die Aufteilung aus einer *Parser-First*-Perspektive angegangen. Hierbei werden alle für die weitere Verarbeitung relevanten Teile im Parser gehalten, damit diese in den entstehenden Syntaxbaum überführt werden können, während der Scanner alle in sich geschlossenen Bausteine nach Möglichkeit über reguläre Ausdrücke zusammenfasst und als atomare Einheit an den Parser übergibt.

Um diese Verteilung zu ermöglichen, muss in der Implementierung für jedes definierte Element betrachtet werden, ob es sich dabei um ein in sich geschlossenes oder atomares Element handelt, welches über reguläre Ausdrücke im Scanner bedient wird, oder ob es ein komplexes Element ist, dessen Zusammensetzung die Auswertung des Ausdrucks beeinflusst.

4.1.3. Anpassungen der Grammatik

Über Umformungen der im RFC definierten Ausgangsgrammatik kann diese an die Implementierungsumgebung angepasst und potenziell stellenweise vereinfacht werden, wodurch die weitere Verarbeitung einfacher und weniger rechenaufwändig gestaltet werden kann. Umformungen der Ausgangsgrammatik stellen jedoch auch eine große potenzielle Fehlerquelle dar, da für RFC-Konformität die von der umgeformten Grammatik akzeptierten und verworfenen Ausdrücke mit denen der Ausgangsgrammatik übereinstimmen müssen.

Um die Konformität zum RFC zu gewährleisten, muss also für jede Umformung überprüft werden, ob diese inhaltlich noch der Ausgangsgrammatik entspricht. Da diese Überprüfung je nach Komplexität der Umformung selbst an Komplexität gewinnen kann, werden Umformungen der Ausgangsgrammatik auf das Nötigste beschränkt.

Daraus ergibt sich das Ziel der in dieser Arbeit verwendeten Umformungen, möglichst nah an der Ausgangsgrammatik zu bleiben und Änderungen systematisch und nachvollziehbar zu gestalten. Hierfür wird im Weiteren zunächst eine Systematik definiert, anhand welcher die Umformungen vollzogen werden können.

Umformungssystematik

Teile der Ausgangsgrammatik, welche ohne Umformungen übernommen werden können, werden ohne Umformungen übernommen. Umformungen werden auf das kleinstmögliche sinnvolle Maß beschränkt.

Grammatikumformungen, die derselben Ursache geschuldet werden beziehungsweise dasselbe Ziel verfolgen, müssen nach demselben Prinzip und mit einheitlicher Namensgebung realisiert werden. Wenn zum Beispiel zwei Regeln eine Stelle derselben Art beinhalten, die für den Parser umgeformt werden muss, so werden beide Stellen nach dem gleichen Schema umgeformt und wo nötig umbenannt. Dieses Schema muss gleichermaßen auch für weitere Stellen dieser Art verwendbar sein.

Abweichungen von einem verwendeten Umformungsschema müssen begründet werden. Wenn ein andersorts verwendetes Umformungsschema auf eine weitere Stelle anwendbar ist, muss dieses andernfalls angewendet werden.

Weitere Umformungsüberlegungen

Im Sinne einer minimalumgeformten Grammatik werden große Teile verbatim aus RFC-9535 übernommen. Um dies erkennbar zu machen und gleichzeitig die Überprüfung beziehungsweise Wartung der Grammatik zu vereinfachen, werden die verwendeten Regeln anhand der Reihenfolge des RFCs notiert und enthalten Verweise auf den jeweiligen Abschnitt des RFC-Dokuments, welchem sie entnommen werden können.

4.1.4. Technologie-Auswahl

Um die Komponenten des Systems möglichst reibungslos zu realisieren, müssen einige Entscheidungen bezüglich der verwendeten Technologien getroffen werden. Diese werden im Folgenden vorgestellt.

Programmiersprache

Für die Wahl der Programmiersprache, in welcher das Gesamtsystem implementiert werden kann, stehen prinzipiell zwei Hauptkandidaten zur Verfügung: *C++* und *M*.

Für den Einsatz der Sprache *M* (ehemals *Mumps*) spricht, dass die NoSQL Datenbank YottaDB auf dieser basiert und somit nativ angesprochen werden kann. *M* findet unter anderem im medizinischen und im Finanzbereich Anwendung.

Der Einsatz von C++ ermöglicht eine einfache Einbindung von C und C++ Ressourcen, allen voran die Generatortools flex und Bison, durch welche der Scanner beziehungsweise der Parser umgesetzt wird. Für C++ existieren Schnittstellen zu YottaDB, sodass dessen Einsatz auch möglich ist.

Außerdem ermöglicht der Einsatz von C++ die Verwendung des *Bison-Syntaxtree-Hacks* [1] und dadurch eine automatisierte Erzeugung von Syntaxbäumen inklusive Visualisierung.

C++ genießt eine weite Verbreitung, wodurch viele Dokumentationen und Hilfsressourcen zur Verfügung stehen.

Da der Fokus dieser Arbeit in der Implementation zunächst auf dem Compiler liegt, da dieser die Grundlage für weitere Verarbeitung und Auswertung darstellt, wird hierfür die Programmiersprache C++ verwendet.

Scanner-Generatortool

Die Umsetzung des Scanners kann manuell beispielsweise als C++-Programm realisiert werden, jedoch erhöht dies nicht nur den Implementationsaufwand, sondern führt auch eine unnötige zusätzliche Fehlerquelle in Form der eigenen Implementation ein. Da keine Funktionalität vom Scanner benötigt wird, die nicht über ein Scanner-Generatortool erreicht werden kann, empfiehlt es sich daher, ein solches zu verwenden.

Prinzipiell gibt es verschiedene Generatortools für Scanner, wie zum Beispiel *lex* und *flex*, jedoch unterscheiden sich diese für die meisten Anwendungsfälle nicht voneinander, weshalb das kostenfreie und weit verbreitete Scanner-Generatortool *flex* verwendet wird.

Standardmäßig ermöglicht flex nur die byteweise Verarbeitung von Eingaben, sodass Unicode beispielsweise UTF-8 codiert nicht vollständig abgebildet werden kann. flex ermöglicht die Abbildung der Standard-7-Bit-ASCII-Zeichen, welche für eine JSONPath-Implementierung mit eingeschränktem Namensraum ausreicht. Die vollständige Abbildung des Unicode-Zeichenraums kann zum Beispiel über besondere Codierung realisiert und auch nachgerüstet werden.

Parser-Generatortool

Wie auch bei der Auswahl von Implementierungsmöglichkeiten für den Scanner, gibt es für Parser geeignete Generatortools, welche die potenzielle Fehlerquelle einer eigenen Implementation reduzieren.

Wohl die bekanntesten hiervon sind *yacc* und *Bison*, wobei sich *Bison* durch seine weit verbreitete und kostenfreie Verfügbarkeit und Aufwärtskompatibilität zu *yacc* als Kandidat hervorhebt. Darüber hinaus ermöglicht die Verwendung von *Bison* den Einsatz des *Bison-Syntaxtree-Hacks* und somit eine einfache Überführung der Eingaben in Syntaxbäume.

Erstellung von Syntaxbäumen

Wie vorausgehend besprochen, wird für die Erstellung der Syntaxbäume die *Bison-Syntaxtree-Hack*-Klasse eingesetzt. Diese bindet sich direkt in den *Bison*-Parser ein und erstellt automatisch während des Parsevorgangs den Syntaxbaum. Weiterhin bietet die Klasse eine Visualisierungskomponente an, durch welche die erstellten Syntaxbäume in einer graphischen Darstellung repräsentiert werden können und somit im Aufbau einfacher nachvollziehbar werden.[\[1\]](#)

Weitere Technologien

Da beispielsweise die Implementierung des Interpreters und einer Syntaxbaum-Optimierung aus Zeitgründen in dieser Arbeit nicht erfolgt ist, sind auch entsprechende Technologieentscheidungen hierfür hinfällig und werden für eine eventuelle Fortsetzung der Arbeit nicht festgelegt.

4.2. JSONPath-Grammatik in EBNF

Um den Aufbau und das Zusammenspiel der Grammatikregeln der in RFC-9535 definierten ABNF besser nachvollziehen zu können und die Regeln näher an die Implementation anzupassen, wurde die ABNF in *Erweiterte Backus-Naur-Form* (EBNF) überführt. EBNF ist näher an der Struktur, die beispielsweise *Bison* für Grammatikregeln vorgibt, sodass anhand der EBNF-Grammatik die Regeln für *Bison* einfacher zu definieren sind.

Die vollständige JSONPath-Grammatik in EBNF findet sich zur Referenz in Anhang A.

Die ABNF-Regeln definieren sowohl die Grammatik als auch die spezifischen Zeichen, die verwendet werden können beziehungsweise müssen. Da die Verarbeitung von syntaktischer Analyse und lexikalischer Analyse in Parser und Scanner getrennt voneinander realisiert werden soll, wurde die EBNF in Grammatikregeln und Zeichenregeln unterteilt. Außerdem ist der Grammatik für normalisierte JSONPath-Ausdrücke ein eigener Abschnitt gewidmet.

Die Zeichenangaben in ABNF geben teilweise keine Groß- oder Kleinschreibung vor, definieren aber dafür in anderen Stellen präzise und ausschließliche Angaben zur erforderlichen Groß- beziehungsweise Kleinschreibung. Dies wurde in der EBNF zur Vereinfachung nicht beachtet, muss jedoch in der letztlichen Implementation wieder mit eingebaut werden.

Die Überführung der ABNF in EBNF wurde in mehreren Iterationen durchgeführt, wobei frühere Iterationen sich teilweise im Versuch, die Regeln zu vereinfachen, weiter von der RFC-Definition entfernt haben.

Letztlich wurden keine sinnvollen Vereinfachungen der Regeln gefunden, die sich nicht negativ auf die RFC-Konformität auswirken, weshalb der gegenteilige Ansatz gewählt wurde und die Grammatik in Struktur und Namensgebung möglichst nahe an der ursprünglichen RFC-Definition gehalten wurde. Dies entspricht auch den Überlegungen zu Grammatik-Umformungen, die in Abschnitt 4.1.3 ausgelegt wurden.

Konkrete Umformungen nach der besprochenen Umformungssystematik sind in der EBNF nicht realisiert, mit der Ausnahme von kleinen Änderungen, wie beispielsweise der Verwendung von Unter- statt Bindestrichen in Regelnamen.

4.3. Controller

Der *Controller* stellt die zentrale Steuereinheit des Systems dar. Seine Aufgaben sind die Entgegennahme von Eingaben vom Benutzer und die Steuerung von deren Auswertung. Insbesondere muss der Controller dabei die Eingaben in auszuwertende JSONPath-Ausdrücke und Steuerparameter unterteilen.

Der Controller kann beliebig viele JSONPath-Ausdrücke erhalten, welche er sequenziell abarbeitet. Hierfür übergibt er den einzelnen JSONPath-Ausdruck dem Scanner und Parser, um diesen auf Wohlgeformtheit zu prüfen und im Falle von Wohlgeformtheit in einen Syntaxbaum zu überführen. Anschließend leitet er den Syntaxbaum an den Interpreter weiter, um die Auswertung des Ausdrucks zu starten. Das Ergebnis des Interpreters gibt der Controller abschließend in geeigneter Form aus.

4.4. flex-Scanner

Der flex-Scanner muss in Absprache mit dem Parser die Eingabe in eine Menge aus vordefinierten Tokens überführen. Prinzipiell kann ein Token für eine beliebige Teilmenge der Eingabe stehen, jedoch stellt die Koordination der Tokens eine Kopplung zwischen Scanner und Parser dar, sodass die Zahl der Tokens nicht unnötig groß werden sollte.

Dementsprechend werden keine Tokens definiert, die ein einzelnes Zeichen in der Eingabe unverändert an den Parser weiterreichen, wie dies beispielsweise für Klammern der Fall ist. Um einzelne Zeichen weiterzureichen, werden diese direkt als Zeichen weitergeleitet und können dementsprechend in den Grammatikregeln auch direkt als Zeichen verwendet werden.

Für Teilmengen der Eingabe, deren Länge fest ist, wie dies zum Beispiel für den Vergleichsoperator `==` der Fall ist, werden entsprechende Tokens definiert, damit Whitespaces zwischen den Operatorzeichen ausgeschlossen werden können

Alle weiteren Tokens repräsentieren komplexere Teilmengen des Eingabetextes, die über reguläre Ausdrücke abgebildet werden. Diese regulären Ausdrücke orientieren sich möglichst eng an den Zeichenregeln der Grammatik, sodass unnötige Fehler durch falsche Umformungen vermieden werden können und die regulären Ausdrücke einfacher nachzuvollziehen und bei Bedarf einfacher zu warten und zu erweitern sind.

4.4.1. Whitespace-Verarbeitung

Da es keine Stellen in der JSONPath-Grammatik gibt, an denen Whitespaces verpflichtend sind, wäre ein möglicher Ansatz, die Grammatik erheblich zu vereinfachen, einfach beispielsweise auf Scanner-Ebene alle Whitespaces zu überlesen. Whitespaces auf Grammatik-Ebene müssen über Optionen in die betroffenen Regeln eingebunden werden, damit diese an allen erlaubten Stellen vorkommen können, ohne vorkommen zu müssen. Dies führt dazu, dass diese Regeln erheblich komplexer werden, wodurch der Parsevorgang fehleranfälliger wird.

Es gibt jedoch einige Stellen, an denen JSONPath Whitespaces nicht erlaubt, welche bei einer vollständigen Überlesung von Whitespaces nicht realisiert werden können.

1	\$. a
2	\$. a
3	\$. a

Das obige Beispiel zeigt einen einfachen JSONPath-Ausdruck und mögliche Anordnungen von Whitespaces. Ein Whitespace zwischen dem Punktoperator und dem darauf folgenden Selektor ist jedoch nicht erlaubt, weshalb der unterste Ausdruck nicht als JSONPath-Ausdruck nach RFC-9535 akzeptiert werden darf.

Der naheliegende Ansatz, die Whitespaces wie im RFC als Teil der Grammatik zu behandeln und entsprechend im Parser darzustellen, erhöht wie bereits beschrieben die Fehleranfälligkeit des Parsers und führt zu diversen SR- und RR-Konflikten. Es ist jedoch in gewisser Weise auch der am wenigsten fehleranfällige Ansatz, da dieser wie auch der RFC über positive Definition die Stellen kennzeichnet, an welchen Whitespaces erlaubt sind, während Stellen, an denen Whitespaces verboten sind, implizit durch Abwesenheit einer Erlaubnis dargestellt werden.

Wenn Whitespaces beispielsweise im Scanner behandelt werden sollen, ist der zuverlässigere Weg hierfür, die Stellen durchzusetzen, an denen keine Whitespaces verwendet werden dürfen. Dementsprechend muss für solche Ansätze dieser Aspekt der Grammatik erfolgreich umgedreht werden, was eine Grammatikänderung als mögliche Fehlerquelle zur Folge hat.

Die Behandlung von Whitespaces im Scanner, also effektiv die Umsetzung von Stellen, an denen Whitespaces verboten sind, erhöht die Komplexität der verwendeten regulären Ausdrücke. Auch unscheinbare Stellen müssen hierbei bedacht werden, so können beispielsweise Operatoren, die aus mehreren Zeichen bestehen, nicht ohne weitere Prüfungen als einzelne Zeichen an den Parser gereicht werden, da sonst Whitespaces zwischen den Operatorzeichen nicht erkannt werden.

Des weiteren geht diese Bearbeitung davon aus, dass der Scanner ausreichende Informationen hat, um entscheiden zu können, ob ein gegebenes Whitespace erlaubt ist oder nicht. Wie sich in der Implementation herausstellen wird, ist dies nicht für alle Stellen der Fall.

Ein weiterer möglicher Ansatz wäre es, Whitespaces in Scanner und Parser zu ignorieren und in einem anschließenden Nachbearbeitungsschritt zu prüfen. Effektiv entspricht das einem zweiten, auf Whitespaces und eventuelle andere Nachbearbeitungen spezialisierten Parsevorgang, was dementsprechend nennenswerte negative Auswirkungen auf die Laufzeit des Compilers haben kann. Ebenso wird die Frage der konkreten Lösung der Whitespaces damit auch nur auf einen Nachbearbeitungsschritt verschoben und muss folglich nach wie vor geklärt werden.

Da die Bearbeitung von Whitespaces im Scanner vergleichsweise einfach umzusetzen ist und dabei die Vorteile für den Parser ohne größere Nachteile für den Scanner erreichen kann, wird diese für die Implementation verwendet. Whitespaces werden somit aus der Grammatik entfernt und beim Auswerten der Tokens beachtet. Dies stellt jedoch einen Kompromiss dar, da nicht alle kritischen Stellen erfolgreich im Scanner abgehandelt werden können.

Es entsteht dadurch an einigen Problemstellen die Möglichkeit, dass JSONPath-Ausdrücke fälschlich als solche akzeptiert werden, obwohl sie nach RFC-9535 keine solchen sind. Die konkret betroffenen Stellen werden in der Implementierung aufgeführt.

4.5. Bison-Parser

Um die syntaktische Analyse mit einem von Bison generierten Parser zu ermöglichen, müssen einige Regelkonstrukte aus der ABNF beziehungsweise EBNF umgeformt werden.

Bison-Regeln können keine Wiederholungsklammern oder Optionsklammern beinhalten, weshalb beide eine passende Umformung benötigen. Um diese Umformungen einheitlich und nachvollziehbar zu halten, werden folgende Umformungsschemata eingesetzt:

4.5.1. Wiederholungsschema

```
1  A = {B}
2
3  A = rep_B
4  rep_B = epsilon
5         | rep_B B
```

Das obige Schema zeigt, wie aus einer EBNF-Regel mit einer Wiederholung durch Umformung die Wiederholung auf eine rekursive Hilfsregel ausgelagert wird. Um ein einheitliches Namensschema zu erhalten, wird die Hilfsregel nach dem sich wiederholenden Teil benannt und erhält den Präfix *rep_* (englisch *repetition*).

```
1  A = B B B rep_B
2  rep_B = epsilon
3         | rep_B B
```

Wiederholungen mit einer festen Mindestanzahl an Wiederholungen müssen zusätzlich zur Hilfsfunktion diesen Mindestanteil als festen Bestandteil der Regel definieren. Obiges Beispiel zeigt eine mindestens dreifache Wiederholung von *B*.

```
1  A = opt_B opt_B opt_B
```

Wiederholungen mit einer festen Höchstanzahl an Wiederholungen müssen anstelle der Wiederholungshilfsregel die benötigte Anzahl über Optionen ausdrücken. Obiges Beispiel zeigt eine maximal dreifache Wiederholung von B .

```
1  A = B B B
```

Wiederholungen mit einer festen Anzahl an Wiederholungen, also derselben festen Mindest- und Höchstanzahl, müssen diese über eine entsprechende Anzahl des Basisausdrucks regeln. Obiges Beispiel zeigt eine genau dreifache Wiederholung von B .

4.5.2. Optionsschema

```
1  A = [B]
2
3  A = opt_B
4  opt_B = epsilon
5  | B
```

Optionsklammern können in Bison nicht direkt dargestellt werden. Um optionale Teile zu realisieren, wird das obige Umformungsschema benutzt. Hierbei wird der optionale Teil durch eine Hilfsregel ersetzt, welche auf diesen oder die leere Menge abgeleitet werden kann. Wie auch beim Wiederholungsschema setzt das Optionsschema auf eine einheitliche Namensgebung, für welche die Hilfsregeln den Präfix *opt_* (*Option*) erhalten.

4.5.3. Epsilon-Ableitungen

Da viele Regeln eine Epsilon-Ableitung in Form einer leeren Option enthalten, wird zur Erkennbarkeit dieser Stellen eine spezielle Hilfsregel *epsilon* ergänzt. Diese enthält keine weiteren Terminale oder Nichtterminale, wodurch sie der leeren Menge entspricht und eine einheitliche Kennzeichnung der leeren Menge in anderen Regeln ermöglicht.

4.5.4. Normalisierte Ausdrücke

Da JSONPath klare Grammatikdefinitionen für die JSONPath-Ausdrucksteilmenge der normalisierten JSONPath-Ausdrücke bereitstellt, werden diese vom Parser auch umgesetzt.

Die Umsetzung der normalisierten Ausdrücke wurde initial motiviert durch ein Missverständnis, da die ABNF im RFC an verschiedenen Stellen nur Groß- beziehungsweise Kleinbuchstaben angibt und aufgrund dessen, dass Groß- und Kleinschreibung in ABNF nicht beachtet wird, die jeweils nicht angegebene Schreibweise impliziert. Wenn die ABNF Groß- und Kleinbuchstaben unterscheiden würde, würden Randfälle entstehen, in denen ein Ausdruck ein normalisierter JSONPath-Ausdruck, aber *kein* allgemeiner JSONPath-Ausdruck ist.

```
1  $['\u000b']
```

Obiges Beispiel zeigt einen solchen Fall, der unter der Annahme, dass die ABNF Groß- und Kleinschreibung unterscheidet, nur als normalisierter Ausdruck erlaubt ist. Die Definition für normalisierte Hex-Character-Escapes erlaubt in diesen nur Kleinbuchstaben, während die Definition für allgemeine Hex-Character-Escapes zwar nur Großbuchstaben angibt, aber durch die Eigenschaften der ABNF auch Kleinbuchstaben implizit erlaubt.

Unter Berücksichtigung der implizierten Groß- beziehungsweise Kleinschreibungen sind die normalisierten JSONPath-Ausdrücke eine echte Teilmenge der allgemeinen JSONPath-Ausdrücke, wodurch die Grammatik für die normalisierten Ausdrücke nicht notwendig ist, um allgemeine JSONPath-Ausdrücke auf Wohlgeformtheit zu überprüfen.

Da diese Erkenntnis jedoch zu einem Zeitpunkt erfolgt ist, an dem normalisierte Ausdrücke bereits erkannt und von allgemeinen Ausdrücken unterschieden werden konnten, wurden diese beibehalten. Somit kann der Parser nicht nur die Wohlgeformtheit von JSONPath-Ausdrücken überprüfen, sondern auch erkennen, wenn der Ausdruck ein normalisierter JSONPath-Ausdruck ist.

4.5.5. Bison-Syntaxtree-Hack

Um die Überführung der Eingabe in einen als Zwischencode geeigneten Syntaxbaum zu automatisieren, wird die Bison-Syntaxtree-Hack-Klasse [1] in den Parser eingebunden.

Da diese Klasse außerdem über eine Visualisierungskomponente verfügt, wird diese über Steuerparameter im Controller zugänglich gemacht.

4.6. Compiler-Tests

Die Compiler-Tests dienen einer automatischen Bestätigung der Funktionstüchtigkeit des Compilers, insbesondere nach eventuellen Änderungen daran. Hierfür werden Testfälle definiert, welche aus einem Ausdruck und der Information, ob dieser wohlgeformt ist, bestehen. Diese können automatisiert dem Compiler vorgesetzt werden, um dessen Ergebnis mit dem erwarteten Ergebnis abzugleichen.

Es ist hierbei im Interesse einer möglichst fehlerfreien und erweiterbaren Anwendung, so viele Szenarien wie möglich abzudecken, mit dem idealisierten Ziel, alle möglichen Szenarien für wohlgeformte und nicht-wohlgeformte JSONPath-Ausdrücke zu beinhalten. Da sowohl die Anzahl der wohlgeformten als auch insbesondere die Anzahl der nicht-wohlgeformten Ausdrücke unendlich ist, ist eine Test-Suite, die vollständig alle möglichen Fälle abdecken kann, nicht realistisch.

Ziel der Tests ist es daher zunächst, eine ungefähre Idee zu geben, ob die aktuelle Compiler-Konfiguration die Wohlgeformtheit von Ausdrücken richtig erkennen kann. Diese Grundlage kann anschließend um weitere Testfälle erweitert werden, um weitere Szenarien abdecken zu können.

4.7. Interpreter

Die Auswertung von JSONPath-Ausdrücken erfolgt segmentweise. Jedes Segment erhält als Eingangswert eine Menge von Knoten aus dem JSON-Objekt, auf welchem die Auswertung ausgeführt werden soll, wählt anhand dieser Knoten und den im Segment definierten Selektoren eine neue Knotenmenge aus und gibt diese als Rückgabewert aus.

Das erste Segment erhält hierbei den Wurzelknoten des JSON-Objekts und die Ausgabe des letzten Segments ist die Ausgabe des gesamten JSONPath-Ausdrucks. Wenn an irgendeiner Stelle die weitergegebene Knotenmenge leer ist, kann die Auswertung vorzeitig abgebrochen werden, da jedes Segment nur aus den noch vorhandenen Knoten und deren Kindknoten eine Auswahl treffen kann.

Aufgrund dieser Beschaffenheit von JSONPath-Ausdrücken muss der Interpreter nicht den gesamten JSONPath-Ausdruck auf einmal auswerten können, sondern kann dessen Segmente der Reihe nach auswerten, um das Gesamtergebnis zu erhalten.

4.8. Beschleunigte Suche in YottaDB

Im Folgenden wird die Schlüsselzerlegung als ein Ansatz zur Beschleunigung der Suche in YottaDB betrachtet und auf ihre Laufzeit und Speicherkomplexität untersucht.

Weitere Ansätze können ungefähr nach demselben Schema auf ihre Eignung untersucht werden. Da jedoch, wie sich zeigen wird, der untersuchte Ansatz in erster Linie an der wahrscheinlich bereits vorhandenen Effizienz der Suche in YottaDB scheitert und die Implementierung dieser Ansätze nicht im Rahmen dieser Arbeit stattgefunden hat, werden an dieser Stelle darüber hinaus keine weiteren Ansätze verfolgt.

```
1 ["Hallo", "Welt", 1]
```

Ein möglicher Ansatz, um die Suche nach exakt übereinstimmenden Schlüsseln in einem YottaDB-Datensatz zu beschleunigen, ist es, die verwendeten Schlüssel in kleinere Schlüssel zu unterteilen. Dies kann dank der hierarchischen Strukturierung der Schlüssel in YottaDB ohne inhaltliche Änderungen am Datensatz umgesetzt werden, da YottaDB die ersten $n - 1$ Elemente des Schlüssel-Wert- n -Tupels als Schlüssel betrachtet und der Anzahl der Schlüssel keine Grenze setzt.

Wenn also die aufrufende Stelle die Zerlegung der Schlüssel kennt, entsteht kein Unterschied zwischen dem obigen Beispiel mit Schlüsseln *Hallo* und *Welt* und folgendem Tupel:

```
1 ["H", "a", "l", "l", "o", "W", "e", "l", "t", 1]
```

Darüber hinaus muss nicht die komplette Schlüsselsequenz zerlegt werden, da alle Schlüssel im Schlüsselraum des ihnen vorgestellten Schlüssels funktionieren. Im Ausgangsbeispiel ist der Schlüssel *Welt* ein Schlüssel im Schlüsselraum von *Hallo*, der auf den Wert 1 verweist. Wenn der Schlüssel *Hallo* in einzelne Schlüssel der Länge 1 zerlegt wird, verweist der Schlüssel *Welt* nach wie vor auf den Wert 1, bezieht sich aber nun auf den Schlüsselraum von *o*.

Auf dieselbe Weise müssen einzelne Schlüssel nicht vollständig unterteilt werden. Wenn beispielsweise ein Schlüsselraum mit zahlreichen Schlüsseln wie *Hallo Welt*, *Hallo Peter*, und so weiter vorhanden ist, reicht es, die Schlüssel in *Hallo* und den restlichen Schlüssel zu unterteilen.

Da die Zerlegung der Schlüssel nach Bedarf entschieden werden kann, muss zunächst geklärt werden, ab wann sich die Zerlegung einer Schlüsselebene lohnt. Um die Komplexität der Betrachtungen etwas zu reduzieren, wird vorerst davon ausgegangen, dass die Schlüsselzerlegung auf einer Ebene ausgeführt wird und diese vollständig in einzelne Zeichen zerlegt wird. Wenn die Zerlegung einer Schlüsselebene einen Laufzeitvorteil bringt, dann lässt sich derselbe Vorteil auch auf anderen Ebenen erzielen, wenn diese ausreichend viele Schlüssel beinhalten.

4.8.1. Vollständige Zerlegung einer Schlüsselebene

Um die Auswirkungen der Zerlegung einer Schlüsselebene auf Laufzeit und Speicherkomplexität bestimmen zu können, werden zunächst einige Variablen definiert:

- die Anzahl n der Schlüssel auf der betrachteten Ebene,
- die Länge m dieser Schlüssel und
- die Größe c des für die Schlüssel verwendeten Zeichensatzes.

Im Ausgangsdatensatz müssen bei linearer Suche im schlimmsten Fall alle n Schlüssel der Ebene durchsucht werden. Wenn davon ausgegangen wird, dass die Laufzeit für die Betrachtung eines Schlüssels konstant ist, also weder von der Schlüssellänge m noch der Größe des verwendeten Zeichensatzes c nennenswert beeinflusst wird, ergibt sich daraus eine lineare Laufzeit von $\mathcal{O}(n)$.

Da YottaDB sortierte Datensätze verwendet, kann wie in Kapitel 3 besprochen wurde, von einer verbesserten Laufzeit von $\mathcal{O}(\log(n))$ ausgegangen werden, welche beispielsweise über binäre Suche erreicht werden kann. Da diese Verbesserung auch auf den geänderten Datensatz zutrifft, wird sie jedoch erst später betrachtet und zunächst von einer Laufzeit von $\mathcal{O}(n)$ ausgegangen.

Im geänderten Datensatz müssen $m - 1$ zusätzliche Ebenen durchsucht werden, welche im schlimmsten Fall jeweils c zu durchsuchende Schlüssel enthalten. Dementsprechend müssen bei linearer Suche im schlimmsten Fall $m * c$ Schlüssel durchsucht werden. Da davon ausgegangen wird, dass der Vergleich eines einzelnen Schlüssels mit dem Zielschlüssel in konstanter Zeit erfolgen kann, bringt die Schlüssellänge von 1 keine Vorteile, jedoch ändert sich dies natürlich entsprechend, sobald diese Annahme wegfallen sollte.

Zusammenfassend lässt sich also festhalten, dass der geänderte Datensatz in der Laufzeit der Suche unabhängig von der Anzahl n der Schlüssel im Ausgangsdatensatz ist. Folglich gilt für jedes n , das größer als $m * c$ ist, dass die Zerlegung der Schlüssel einen Vorteil für die Laufzeit darstellt.

Der Speicherbedarf im Ausgangsdatensatz für n Schlüssel der Länge m lässt sich als $n * m$ zusammenfassen. Wenn davon ausgegangen wird, dass YottaDB für jeden Schlüssel denselben Speicheroverhead o benötigt, ist der Overhead für n Schlüssel $n * o$. Daraus ergibt sich ein Gesamtspeicherbedarf von $n * m + n * o$.

Da für den geänderten Datensatz aus jedem Zeichen in einem Schlüssel im Ausgangsdatensatz ein Knoten mit diesem Zeichen geformt wird, liegt der Speicherbedarf ohne Overhead ebenfalls bei $n * m$. Wenn einzelne dieser Knoten zusammenlaufen, wie dies beispielsweise der Fall bei mehreren Schlüsseln mit demselben Anfangsbuchstaben ist, kann sich der Speicherbedarf weiter reduzieren.

Der Overhead im geänderten Datensatz steigt jedoch mit der Anzahl der einzelnen Knoten zu $n * m * o$. Insgesamt ergibt sich also ein Speicherbedarf für den geänderten Datensatz von $n * m + n * m * o$.

Die tatsächliche erwartete Auswirkung kann an einem Rechenbeispiel nachvollzogen werden:

1	$n = 5.000$	// 5.000 zu durchsuchende Schlüssel
2	$m = 16$	// Schlüssel der Länge 16
3	$c = 127$	// 7-Bit-ASCII-Zeichensatz
4	$o = 2$	// Overhead der Größe 2
5		
6	Worst Case Laufzeit im Ausgangsdatensatz:	
7	$n = 5.000$	
8		
9	Worst Case Laufzeit im geänderten Datensatz:	
10	$m * c = 2.032$	
11		
12		
13	Erwarteter Speicherbedarf im Ausgangsdatensatz:	
14	$n * m + n * o = 90.000$	
15		
16	Erwarteter Speicherbedarf im geänderten Datensatz:	
17	$n * m + n * m * o = 240.000$	

Wie aus dem Rechenbeispiel erkennbar wird, bietet die Schlüsselzerlegung signifikante Vorteile in der Laufzeit, welche durch akzeptierbare Einbußen in der Speicherkomplexität erkaufte werden.

4.8.2. Anpassung an verbesserte Laufzeit

Wie in Kapitel 3 besprochen, ist die Annahme, dass YottaDB eine lineare Suche einsetzt und dadurch eine Laufzeit von $\mathcal{O}(n)$ erreicht, nicht unbedingt korrekt. Es muss daher die Auswirkung auf die Laufzeit betrachtet werden, wenn beispielsweise über binäre Suche die Dauer einer einzelnen Suche auf einer Ebene auf $\mathcal{O}(\log(n))$ reduziert wird.

Die Änderungen an der betrachteten Laufzeit haben keinen Einfluss auf die Speicherkomplexität, sodass diese im weiteren nicht mehr betrachtet werden muss. Allein nach der Speicherkomplexität ist die Schlüsselzerlegung ein erhöhter Speicheraufwand, der jedoch für die meisten Datensätze problemlos in Kauf genommen werden kann.

Im Ausgangsdatensatz ändert sich die Zahl der betrachteten Einträge von n zu $\log(n)$.

Der geänderte Datensatz muss nach wie vor m Ebenen durchsuchen, kann jedoch die einzelnen Ebenen ebenso von c maximal betrachteten Einträgen auf $\log(c)$ Einträge reduzieren. Daraus ergibt sich eine Gesamtlaufzeit im geänderten Datensatz von $m * \log(c)$.

Dementsprechend lohnt sich der Einsatz der Schlüsselzerlegung nur, wenn $\log(n)$ größer ist als $m * \log(c)$. Unter der Annahme, dass binäre Suche zum Einsatz kommt und der verwendete Logarithmus dementsprechend $\log_2$ ist, reduziert sich im vorigen Rechenbeispiel die Laufzeit im Ausgangsdatensatz auf circa 12, während sich die Laufzeit beim geänderten Datensatz nur auf circa 112 reduziert. Bei $n = 5.000$ betrachteten Schlüsseln lohnt sich dementsprechend der Aufwand, den Datensatz umzuformen, nicht mehr.

Um zu bestimmen, ab wann sich die Schlüsselzerlegung unter Anbetracht der verbesserten Suchlaufzeit lohnt, kann folgende Gleichung betrachtet werden:

$$\log_2(n) > m * \log_2(c)$$

$$n > 2^{m * \log_2(c)}$$

Beim vorigen Rechenbeispiel mit Schlüsseln der Länge 16 und einem Zeichensatz der Größe 127 ergibt sich daraus circa $n > 5,2 * 10^{33}$, also mit anderen Worten müssten mehr als 5,2 *Quintilliarden* Schlüssel im Ausgangsdatensatz zu durchsuchen sein. Die vollständige Zerlegung einer Schlüsselebene bringt also keinen realistischen Mehrwert gegenüber der wahrscheinlich von YottaDB eingesetzten binären Suche.

Um die Schlüsselzerlegung möglicherweise konkurrenzfähig zu machen, müssten idealerweise sowohl die Schlüssellänge m als auch die Größe c des verwendeten Zeichensatzes verkleinert werden. Beide dieser Ansätze werden im Folgenden weiter untersucht.

4.8.3. Reduktion des Zeichensatzes

Eine Reduktion des verwendeten Zeichensatzes ist problemlos möglich, wenn der ursprüngliche Zeichensatz nicht vollständig ausgeschöpft wird. Wenn zum Beispiel der Ausgangsdatensatz von einem 7-Bit-ASCII-Zeichensatz ausgeht, aber Schlüssel tatsächlich nur aus Groß- und Kleinbuchstaben bestehen, würde die Einschränkung des Zeichensatzes auf diese Buchstaben keine Änderungen an den bestehenden Schlüsseln erfordern, könnte aber bereits nennenswerte Einsparungen in der Laufzeit bewirken.

Wenn der Datensatz zukünftig erweiterbar bleiben soll, müssten neue Einträge entweder auch zur Verwendung des reduzierten Zeichensatzes gezwungen werden, oder die Reduktion müsste um die neuen Zeichen zurückgenommen werden und die Laufzeit entsprechend verschlechtert werden.

Eine Reduktion des Zeichensatzes jenseits der tatsächlich verwendeten Zeichen ist möglich und kann weitere Ersparnisse bringen. Beispielsweise könnten Schlüssel, die nur aus Buchstaben bestehen, auf entweder nur Groß- oder nur Kleinbuchstaben reduziert werden. Jedoch können hierbei Konflikte zwischen Schlüsseln entstehen, wenn die Schlüssel bewusst abhängig von Groß- und Kleinschreibung gewählt wurden.

Im Falle einer Kollision müssten die kollidierenden Schlüssel durch zusätzliche Zeichen unterscheidbar gemacht werden, wodurch sich die Schlüssellänge erhöhen würde.

Da jedoch die Schlüssellänge einen deutlich größeren Einfluss auf die Gesamtlaufzeit hat als der verwendete Zeichensatz, ist eine Reduktion der Schlüssellänge einer Reduktion der Größe des Zeichensatzes im Zweifelsfall vorzuziehen, oder im Idealfall die Reduktion beider Größen zu kombinieren.

4.8.4. Reduktion der Schlüssellänge

Eine direkte Reduktion der Schlüssellänge ist nicht ohne weiteres auf jeden Fall anwendbar, da die Ausgangsschlüssellänge durch den gewünschten Einsatz der Schlüssel im Ausgangsdatensatz bedingt sein kann. Vielmehr ist die zu reduzierende Einheit nicht die tatsächliche Schlüssellänge, sondern die effektive Schlüssellänge für die Schlüsselzerlegung, oder mit anderen Worten der Anteil der Schlüssellänge, der zerlegt werden soll.

Die bisherigen Rechnungen und Laufzeit- und Speicherkomplexitätsüberlegungen bezogen sich auf die vollständige Zerlegung einer Schlüsselebene, jedoch schießt diese schnell über das eigentliche Ziel hinaus. Wenn beispielsweise durch die Zerlegung des ersten Zeichens der Schlüssel im Ausgangsdatensatz bereits die entstehenden Suchräume zu klein werden, um weitere Zerlegung zu rechtfertigen, lohnt es sich, die Schlüssel nicht weiter zu zerlegen, da die weitere Zerlegung die Laufzeit und Speicherkomplexität nur noch negativ beeinflusst.

In der Theorie ist es also möglich, durch gezielte Zerlegung der Schlüssel die Suchräume ideal aufzuteilen, um die erzielte Suchbeschleunigung zu maximieren. Da es jedoch möglich ist, dass selbst unter optimalen Bedingungen die Schlüsselzerlegung keinen Laufzeitgewinn gegenüber der binären Suche bringt, wird im weiteren zuerst abgewogen, ob eine minimale Schlüsselzerlegung sinnvoll sein kann, bevor weitere Überlegungen zum optimalen Maß der Schlüsselzerlegung und dessen Erkennung angestellt werden.

Um zu prüfen, ob eine minimale Schlüsselzerlegung einen Vorteil gegenüber binärer Suche auf dem Ausgangsdatensatz bringen kann, wird hierfür zunächst der Idealfall betrachtet. Konkret bedeutet das, dass der Ausgangsdatensatz n Schlüssel beinhaltet, welche mit ihrem ersten Zeichen den vollen Umfang des verwendeten Zeichensatzes ausschöpfen und die Belegung der einzelnen Zeichen gleich verteilt ist, also jedes Anfangszeichen genau gleich häufig in der Schlüsselmenge auftritt. Dadurch bringt die Zerlegung der Schlüssel anhand des ersten Zeichens folgenden Effekt:

Der Suchraum der Ausgangsebene wird von n Schlüssel im Ausgangsdatensatz zu c Schlüssel im geänderten Datensatz reduziert. Neben der Ausgangsebene muss eine weitere Ebene unter dieser durchsucht werden, auf welcher sich zu jedem Knoten in der Ausgangsebene weitere n/c zu durchsuchende Schlüsselreste befinden.

Daraus ergibt sich für die Laufzeit unter der Annahme, dass auf jeder Ebene binäre Suche oder ein ähnlicher Algorithmus zum Einsatz kommt, ein schlimmster Fall von $\log(c)$ auf der Ausgangsebene und ein zusätzlicher Aufwand von $\log(n/c)$ auf der neuen Ebene. Insgesamt ist die Laufzeit im geänderten Datensatz also $\log(c) + \log(n/c)$.

Folglich lässt sich die Effizienz der minimalen Schlüsselreduktion anhand dieser Gleichungen überprüfen:

$$\log_2(n) > \log_2(c) + \log_2(n/c)$$

$$n > 2^{\log_2(c) + \log_2(n/c)}$$

$$n > 2^{\log_2(c)} + 2^{\log_2(n/c)}$$

$$n > c * n/c$$

$$n > n \quad \text{!}$$

Wie aus den obigen Gleichungen erkennbar wird, bringt die minimale Schlüsselzerlegung in ihrem Idealfall keinen Vorteil gegenüber binärer Suche. Folglich sind weitere Überlegungen zur Auswahl des idealen Zerlegungsbereichs hinfällig, da die Anwendung der minimalen Schlüsselzerlegung bereits keinen Vorteil bringt und dementsprechend die Anwendung weiterer Schlüsselzerlegungen in den Restschlüsseln ebenso keinen Vorteil erzielen kann.

Die Schlüsselzerlegung bringt dementsprechend unter der Annahme, dass in YottaDB binäre Suche oder ein anderer Algorithmus mit logarithmischer Laufzeit zum Einsatz kommt, keinen Laufzeitvorteil und ist daher nicht zur Beschleunigung der Suche in YottaDB geeignet.

5. Implementierung

Nachfolgend werden die in Kapitel 4 dargestellten Lösungsansätze in eine Implementierung überführt. Hierbei werden verschiedene Implementierungsdetails illustrativ herausgegriffen und genauer betrachtet.

Da es im Umfang dieser Arbeit nicht möglich war, die Implementation des JSONPath-Interpreters umzusetzen, werden dieser und mit ihm zusammenhängende Komponenten in diesem Kapitel nicht weiter verfolgt.

5.1. Gesamtsystem

Das Gesamtsystem ist entsprechend des Architekturdiagramms als ein C++ Programm realisiert. Da für die Verwendung von flex und Bison die jeweiligen Dateien zunächst in C- beziehungsweise C++-Programme überführt werden müssen, damit diese dann im Hauptprogramm aufgerufen werden können, enthält der Projektordner zusätzlich ein Makefile, welches die Kompilierung des Gesamtsystems automatisiert.

5.2. Controller

Der Controller beziehungsweise die zentrale Steuereinheit des Systems wurde als C++-Konsolenprogramm realisiert. Dadurch lässt er sich sowohl von einem Nutzer manuell ansteuern, als auch über automatisierte Aufrufe in einen Programmablauf einbinden. Um Zugriff auf den Zwischencode zu erhalten, muss dieser jedoch nach aktuellem Stand auf Programmseite umgesetzt werden.

Für Aufrufe des Controllers werden diverse Betriebsmodi und Steuersignale unterstützt. Die möglichen Betriebsmodi sind die Verarbeitung eines beim Aufruf mitgegebenen JSONPath-Ausdrucks, die Batch-Verarbeitung einer beliebigen Anzahl beim Aufruf mitgegebener JSONPath-Ausdrücke und die aktive Eingabe.

Wenn kein JSONPath-Ausdruck im Aufruf des Systems enthalten ist, wird dieses im aktiven Modus gestartet und stellt eine Eingabemaske, welche einzelne JSONPath-Ausdrücke entgegennimmt und überprüft. Um die Maske zu verlassen, kann diese durch den Steuerbefehl *quit* beziehungsweise *exit* verlassen werden. Alle anderen Eingaben werden als möglicher JSONPath-Ausdruck interpretiert und auf Wohlgeformtheit untersucht.

Neben den Betriebsmodi bietet der Aufruf des Systems einige Steuersignale und deren Kurzformen, um verschiedene Verhalten auszulösen:

- *-debug* beziehungsweise *-d* zur Aktivierung der Bison-Debug-Option
- *-run-tests* beziehungsweise *-rt* zur Ausführung der Compiler-Tests
- *-verbose* beziehungsweise *-v* zur Aktivierung der Bison-Parser-Verbose-Option
- *-visualize* beziehungsweise *-vis* zur Aktivierung der Visualisierungskomponente der Bison-Syntaxtree-Hack-Klasse

Die Steuersignale können an beliebiger Stelle im Aufruf verwendet werden und beziehen sich auf den gesamten Aufruf. Das Steuersignal *-run-tests* führt die Compiler-Tests aus und beendet anschließend das Programm, wodurch eventuelle weitere Angaben im Aufruf nicht ausgewertet werden.

5.3. flex-Scanner

Der flex-Scanner ist nur für den Standard-7-Bit-ASCII-Zeichensatz definiert, da ein größerer Zeichensatz eine gesonderte Behandlung der Eingabe wie zum Beispiel eine spezielle Codierung erfordern würde. In Bezug auf die Funktionsfähigkeit für JSONPath bedeutet dies, dass der Namensraum in den verarbeitbaren JSONPath-Ausdrücken auf 7-Bit-ASCII-Zeichen eingeschränkt wird.

Die Eingabe wird anhand diverser regulärer Ausdrücke in Tokens unterteilt. Alle Eingaben, die nicht auf einen dieser Tokenausdrücke passen, werden als einzelnes Zeichen dem Parser übergeben. Ausnahmen hiervon stellen zum einen Whitespaces dar, da diese im Scanner überlesen werden und zum anderen Zeichen unterhalb von ASCII `\x20` (Leerzeichen) und überhalb von ASCII `\x7F`, da diese entweder nicht in JSONPath-Ausdrücken auftreten dürfen oder außerhalb des 7-Bit-ASCII-Bereichs liegen und daher für den Scanner nicht definiert sind.

5.3.1. Whitespaces

Wie bereits erwähnt werden Whitespaces vom Scanner überlesen, um den Parser zu entlasten. Dementsprechend muss der Scanner anderweitig sicherstellen, dass alle Stellen eingehalten werden, an denen keine Whitespaces erlaubt sind.

Whitespaces vor beziehungsweise nach dem Gesamtausdruck werden hierbei ignoriert, da diese zwar im RFC nicht vorgesehen sind, aber auch keine Änderungen am Ausdruck vornehmen. Prinzipiell sollten diese Whitespaces nicht als Teil des Gesamtausdrucks betrachtet werden, insbesondere bei einem normalisierten Ausdruck, jedoch wird ihre Entfernung bei Bedarf auf den aufrufenden Kontext des Compilers ausgelagert.

Konkret werden diese zum Beispiel automatisch entfernt, wenn das Programm mit JSONPath-Ausdrücken im Aufruf gestartet wird, da die einzelnen Ausdrücke von Konsolenprogrammen standardmäßig durch Leerzeichen getrennt werden und diese nicht Teil der Aufrufparameter sind.

Um Whitespaces zwischen dem Punkt- oder Nachfolgeoperator und dem darauf folgenden Selektor zu unterbinden, verwenden die regulären Ausdrücke für diese Operatoren einen Lookahead. Im Umkehrschluss bedeutet das, dass bei einem Whitespace zwischen Operator und Selektor der Operator nicht als passendes Token zurückgeliefert wird, sondern als einzelnes Zeichen, und somit von der Grammatik nicht akzeptiert wird.

Um Whitespaces in Zahlen zu verhindern, werden diese auch weitestmöglich über reguläre Ausdrücke gehandhabt. Spezifisch werden Integer hierfür inklusive ihrer eventuellen Vorzeichen definiert, sodass keine Whitespaces zwischen Vorzeichen und Ziffern erlaubt werden.

Folglich muss auch für den Randfall -0 , welcher eine Zahl, aber kein Integer darstellt, dieser als gesondertes Token betrachtet werden. Ebenso müssen Whitespaces zwischen Ziffern und Dezimalpunkt beziehungsweise Exponentoperator ausgeschlossen werden, was über Startbedingungen realisiert wird.

Whitespaces zwischen Funktionsnamen und der öffnenden Funktionsklammer werden über einen Lookahead ausgeschlossen. Dadurch können Funktionsnamen gefolgt von Whitespaces nur als Schlüsselwörter erkannt werden, falls die Funktion den Namen eines Schlüsselwortes hat, in welchem Fall die nach den Whitespaces folgende Funktionsklammer für das Scheitern des Ausdrucks sorgt.

Whitespaces innerhalb von singulären Abfragen sind nicht erlaubt, können jedoch über keine der genannten Methoden abgefangen werden. Um diese Whitespaces gemäß RFC-9535 zu unterbinden, muss dies in einem Nachbearbeitungsschritt nach dem Compilevorgang getan werden.

Da das Ergebnis des Compilevorgangs in Form des Syntaxbaums keine Whitespaces enthält, muss für die Einhaltung des RFCs der Syntaxbaum traversiert werden und bei erkannten singulären Abfragen erneut der ursprüngliche Eingabetext an der passenden Stelle auf Whitespaces geprüft werden.

5.3.2. Schlüsselworte und Operatoren

Die Schlüsselworte *true*, *false* und *null*, sowie alle Operatoren, die aus mehreren Zeichen bestehen, wie beispielsweise das logische Und und Oder, werden als gesonderte Tokens weitergereicht. Dies ermöglicht es, Whitespaces innerhalb dieser Operatoren zu unterbinden.

Da vier von sechs der Vergleichsoperatoren auf diese Weise gesondert betrachtet werden, werden auch die Vergleichsoperatoren *größer* und *kleiner* tokenisiert, obwohl diese nur ein Zeichen lang sind.

Über die Tokenisierung der Schlüsselwörter wird auch deren erforderliche Kleinschreibung durchgesetzt.

5.3.3. Komplexe reguläre Ausdrücke

Für die Tokenisierung komplexerer Einheiten, wie beispielsweise die der String-Literale, werden diese über reguläre Ausdrücke abgebildet, die sich an den zugehörigen Zeichenregeln der EBNF-Grammatik orientieren. Insbesondere String-Literale werden hierfür in kleinere Komponenten unterteilt, die jeweils eine Regel repräsentieren, da ein einzelner großer regulärer Ausdruck schwieriger zu lesen und entsprechend schwieriger zu warten ist.

Tokens, die neben ihrer identifizierenden Aussage auch Inhalte übermitteln müssen, definieren diese anhand des Typs entweder als Character-Array (C-String) oder als Integerwert. Diese können vom Scanner mit dem entsprechenden Wert befüllt werden.

Hierbei kann bei Zahlen der Fraktionsteil beziehungsweise Exponent bereits als dessen Zahlenwert dargestellt werden, da die Identifikation über das entsprechende Token erfolgt, also würde zum Beispiel 1.0 als Tokenfolge einen Integer 1 gefolgt von einem Fraktionsteil 0 erkennen.

Obwohl Integerwerte aus dem Parser ausgelagert werden können, ist es nicht möglich, Zahlen vollständig auszulagern, da nicht über das längste Match zwischen Zahl und Integer unterschieden werden kann, wenn die Zahl weder Fraktionsteil noch Exponent hat.

Dennoch muss diese Unterscheidung für die Auswertung im Parser verfügbar sein, weshalb der Parser Zahlen aus mehreren Tokens zusammenbauen muss. Ein Sondertoken, welches nur für Zahlen relevant ist, ist dabei die Minusnull, da diese kein erlaubter Integer ist, aber als Zahl verwendet werden darf.

5.3.4. Startbedingungen

Standardmäßig wertet flex zu jeder Zeit alle verfügbaren regulären Ausdrücke aus, um unter diesen das längste gefundene Match zu wählen. Um reguläre Ausdrücke zu definieren, die nicht immer beachtet werden dürfen, bietet flex die Möglichkeit, über sogenannte Startbedingungen den Zustand des Scanners zu steuern und diese regulären Ausdrücke an einen Zustand zu binden.

flex unterscheidet hierbei zwischen exklusiven und inklusiven Startbedingungen. Eine exklusive Startbedingung bedeutet, dass, wenn diese aktiv ist, nur reguläre Ausdrücke betrachtet werden, die an diese Bedingung gekoppelt sind. Eine inklusive Startbedingung betrachtet nach wie vor alle regulären Ausdrücke ohne explizite Bedingung, aktiviert aber zusätzlich die an diese Bedingung gebundenen Ausdrücke.

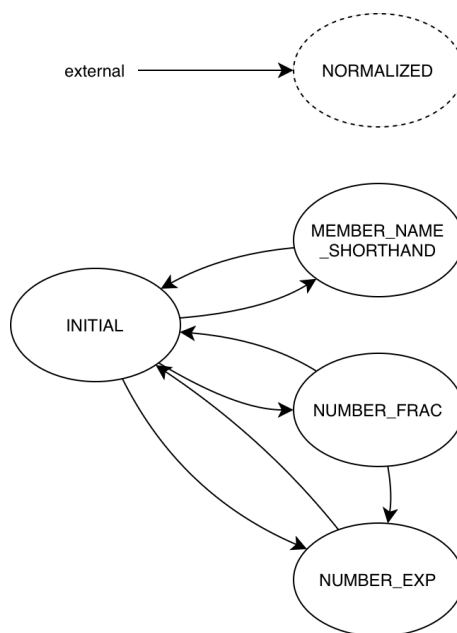


Abbildung 5.1.: Zustandsdiagramm der Startbedingungen

Abbildung 5.1 zeigt die im Scanner verwendeten Startbedingungen als ein Zustandsdiagramm. Da eine inklusive Startbedingung den Neutralzustand *INITIAL* inhaltlich nicht verlässt, sondern lediglich erweitert, werden diese gestrichelt eingezeichnet. Der Scanner kann sich jedoch zu jeder Zeit in genau einem Zustand aufhalten, sodass inklusive Startbedingungen auch für das Zustandsdiagramm von Bedeutung sind.

INITIAL-Zustand

Der *INITIAL*-Zustand ist ein von flex definierter Neutralzustand. Wenn keine Startbedingungen verwendet werden, befindet der Scanner sich in diesem Zustand. Um eine aktivierte Startbedingung zu deaktivieren, kann in diesen Zustand zurückgewechselt werden.

MEMBER_NAME_SHORTHAND-Zustand

Der *MEMBER_NAME_SHORTHAND*-Zustand ermöglicht eine kontextabhängige Verarbeitung von Member-Name-Shorthands. Diese ist nötig, um zwischen Schlüsselwörtern wie *true*, *false* und *null* und Member-Name-Shorthands zu differenzieren, ohne dabei letztere einzuschränken. Viele Programmiersprachen erlauben nicht die Benennung von Variablen nach Schlüsselwörtern, jedoch macht JSONPath keine solchen Einschränkungen, weshalb auch die Implementation diese Einschränkung nicht machen sollte.

```
1 $.true[?@ == true]
```

Das obige Beispiel zeigt ein mögliches Szenario, in welchem *true* sowohl als Schlüsselwort als auch als Name verwendet wird. Um derartige Szenarien korrekt zu tokenisieren, muss der Member-Name-Shorthand-Erkennung ein Kontext angehaftet werden.

Der *MEMBER_NAME_SHORTHAND*-Zustand wird aktiviert, wenn der Scanner einen auf einen Punkt- oder Nachfolgeroperator folgenden Text erkennt, und bleibt nur für dessen Verarbeitung aktiv.

NUMBER_FRAC- und NUMBER_EXP-Zustand

Die Zustände *NUMBER_FRAC* und *NUMBER_EXP* dienen der Beachtung von Whitespaces bei der Verarbeitung von Zahlen. Spezifisch unterdrücken sie durch die Einhaltung einer genau definierten Abfolge Whitespaces zwischen Ziffern und Punkten beziehungsweise zwischen Ziffern und Exponentoperatoren.

Dementsprechend müssen diese Zustände alle möglichen Szenarien für Zahlen abdecken. So wird der *NUMBER_FRAC*-Zustand aktiviert, wenn auf eine Ziffer ein Punkt folgt und der *NUMBER_EXP*-Stand wird aktiviert, wenn auf eine Ziffer ein Exponentoperator folgt. Letzteres ist sowohl aus dem *INITIAL*-Zustand als auch aus dem *NUMBER_FRAC*-Zustand möglich.

Beide Zustände heben sich selbst nach ihrer Abarbeitung auf, wobei der *NUMBER_FRAC*-Zustand wie bereits erwähnt entweder in den *NUMBER_EXP*-Zustand oder den *INITIAL*-Zustand weiterleitet.

NORMALIZED-Zustand

Der *NORMALIZED*-Zustand ermöglicht die Verarbeitung von der eingeschränkten Grammatik für normalisierte Ausdrücke. Da diese Startbedingung inklusiv ist, werden die anderen regulären Ausdrücke nach wie vor berücksichtigt, jedoch machen diese keinen Unterschied für die Auswertung, da ihre Tokens in der normalisierten Grammatik nicht akzeptiert werden.

Spezifisch löst der *NORMALIZED*-Zustand das Problem, dass zwischen reduzierter und allgemeiner Stringsyntax beziehungsweise zwischen reduzierten und allgemeinen Indizes unterschieden werden muss, diese Unterscheidung jedoch nur im Falle der Verarbeitung von normalisierten Ausdrücken getroffen werden muss.

Beispielsweise kann der String 'a' sowohl als normalisierter String als auch als allgemeiner String interpretiert werden. Damit also die allgemeine Regel auch diesen String erkennen kann, muss diese entweder auch normalisierte Strings akzeptieren, oder der String muss als allgemeiner String tokenisiert werden.

Da der *NORMALIZED*-Zustand nur bei der Verarbeitung von normalisierten Ausdrücken benötigt wird, wird dieser Zustand durch den Controller von außerhalb des Scanners gesteuert. Hierfür stellt der Scanner entsprechende Hilfsmethoden zur Verfügung, um den Zustand für den Controller sichtbar zu machen.

5.4. Bison-Parser

Der Bison-Parser definiert anhand der JSONPath-Grammatik in ABNF aus RFC-9535 beziehungsweise der Grammatik in EBNF die nötigen Regeln, durch welche die Grammatik bis auf die Tokenebene abgebildet werden kann.

Um Verwendungen einer leeren Option einheitlich zu halten und diese als bewusste solche erkennbar zu machen, wird eine Zusatzregel *epsilon* verwendet, die auf die leere Menge reduziert.

Wie im Lösungskonzept beschrieben werden Grammatikregeln mit Wiederholungen beziehungsweise Optionsklammern entsprechend der Schemata umgeformt. Ebenso werden durch Kommentare die jeweiligen Abschnitte des RFCs gekennzeichnet, aus welchen die Regeln entnommen werden können, damit nachvollziehbar ist, warum sie ihre Form haben und sie überprüfbar sind.

Als eine Ausnahme zur Umformung von Wiederholungen in Regeln wird zum Beispiel die *segments*-Regel nicht nach dem beschriebenen Schema umgeformt beziehungsweise umbenannt, da sie bereits eine Hilfsregel für die Wiederholung von Segmenten ist und daher nicht um eine weitere Hilfsregel erweitert werden muss.

5.4.1. Normalisierte Ausdrücke

Um normalisierte Ausdrücke erkennen zu können, wird ein alternativer Einstieg in die Grammatik benötigt. Hierfür kann die Startregel *jsonpath* anhand eines vor dem Parsevorgang hinzugefügten Anfangszeichens entscheiden, ob sie die *jsonpath_query*-Regel oder die *normalized_path*-Regel weiterverfolgen soll. Das Steuerzeichen kann dabei entweder ein *j* für allgemeine JSONPath-Ausdrücke oder ein *n* für normalisierte JSONPath-Ausdrücke sein.

Dementsprechend kann der Parser den zu prüfenden Ausdruck doppelt durchlaufen, da zunächst auf einen normalisierten Ausdruck und anschließend auf einen allgemeinen Ausdruck geprüft wird. Sollte die Prüfung auf normalisierte Ausdrücke beispielsweise aus Laufzeitgründen nicht gewünscht werden, lässt sich diese einfach weglassen, ohne dadurch eine Änderung am Gesamtergebnis auszulösen.

5.4.2. Mehrdeutigkeiten in der Grammatik

```
1  $[?$.a]
2  $[?$.a == 1]
```

Es gibt in der JSONPath-Grammatik eine ungelöste Mehrdeutigkeit zwischen singulären Abfragen und JSONPath-Abfragen. Das obige Beispiel zeigt mit \$.a eine Teilsequenz des Ausdrucks, welche sowohl eine singuläre Abfrage als auch eine allgemeine Abfrage sein kann.

Im ersten Fall wird die Sequenz in einem Existenztest verwendet und ist somit als allgemeine JSONPath-Abfrage zu verstehen. Im zweiten Fall wird die Sequenz als Operand eines Vergleichs verwendet und ist somit eine singuläre Abfrage.

An der Stelle des zweiten Dollarzeichens im Beispiel ist es nicht möglich zu erkennen, welche der Optionen hier verfolgt werden muss, da selbst mit einem Lookahead nicht die potenziell beliebig langen JSONPath-Ausdrücke abgedeckt werden können.

Ein deterministischer LR-Parser, wie ihn Bison normalerweise erstellt, verfolgt daher einen der beiden Pfade willkürlich und kann den jeweils anderen dementsprechend nicht auswerten, weshalb nur einer von beiden Fällen korrekt erkannt werden kann.

Um diese Mehrdeutigkeit zu lösen, kann ein GLR-Parser verwendet werden, da dieser beide Möglichkeiten durchläuft und somit die korrekte findet. Jedoch ist die für die Erstellung des Syntaxbaums verwendete Klasse Bison-Syntaxtree-Hack nicht mit GLR-Parsern kompatibel.

Der Parser kann also, solange er den Zwischencode über diese Klasse automatisiert erzeugt, nicht alle JSONPath-Ausdrücke abdecken, beziehungsweise kann, wenn er zur Prüfung der Wohlgeformtheit aller JSONPath-Ausdrücke verwendet werden soll, diese nicht in Zwischencode überführen.

Dementsprechend ist die Zeile der Bison-Datei, die den Parser zu einem GLR-Parser macht, auskommentiert, damit der Parser ins Gesamtsystem eingebunden werden kann. Wenn der Parser mit Mehrdeutigkeiten umgehen können soll, kann diese Zeile wieder einkommentiert werden. In diesem Modus erzeugt der Parser jedoch keinen Syntaxbaum, sondern kann lediglich Ausdrücke akzeptieren oder verwerfen.

5.5. Automatisierte Syntaxbaumerstellung mit Bison-Syntaxtree-Hack

Um automatisiert während des Parsevorgangs einen entsprechenden Syntaxbaum zu erstellen, wird die Klasse Bison-Syntaxtree-Hack in den Parser eingebunden. Dadurch erhält der Controller in der globalen *root*-Variable im Erfolgsfall nach dem Parsevorgang Zugriff zum Syntaxbaum und kann diesen weiter auswerten.

5.5.1. Visualisierung

Die erstellten Syntaxbäume können außerdem mit der Visualisierungskomponente der Bison-Syntaxtree-Hack-Klasse in LaTeX-Form visualisiert werden. Um diese direkt sichtbar zu machen, erzeugt der Controller, wenn die Visualisierung aktiv ist, automatisch die LaTeX-Ausgabe, um diese in eine Datei zu überführen und anschließend zu einer PDF-Datei zu übersetzen.

Hierbei wird als Name der Datei der Erstellungszeitpunkt in Microsekunden als Zeitstempel verwendet, sodass alle Dateien eindeutig benannt sind. Die LaTeX-Datei und alle beim Übersetzen entstandenen Hilfsdateien werden anschließend wieder entfernt, um den Projektordner nicht zu überfüllen.

Die erstellten PDF-Dateien werden über das Makefile mit *make clean* entfernt, sollten also bei Bedarf vorher in einen anderen Ordner verschoben werden.

Um eine Vorstellung der visualisierten Syntaxbäume zu ermöglichen, finden sich in Anhang B drei beispielhafte Syntaxbäume zu folgenden JSONPath-Ausdrücken:

```
1  $['a'][1]
2  $.a[1]
3  $[?@.a > 1 && @.b <= 0]
```

Wie an den ersten beiden Beispielen ersichtlich ist, erkennt der Parser den normalisierten ersten Ausdruck als solchen und stellt dies auch im Syntaxbaum dar.

Der dritte Beispielausdruck zeigt, dass die Syntaxbäume schnell unübersichtlich werden können. Hierbei wäre für die Visualisierung sinnvoll, den Baum in einem Nachbearbeitungsschritt zu traversieren und zu vereinfachen.

Beispielsweise kann in jedem Syntaxbaum der Wurzelknoten auf den entsprechenden *jsonpath_query*- beziehungsweise *normalized_query*-Knoten gesetzt werden, da die oberste *jsonpath*-Ebene nur technisch bedingt ist. Ebenso können zum Beispiel nicht verwendete Wiederholungen und Klammern im Baum weggelassen werden, da diese für den Syntaxbaum keinen inhaltlichen Beitrag leisten.

6. Evaluierung

Im Folgenden wird die Implementation anhand der in Kapitel 3 definierten Anforderungsliste evaluiert, um zu zeigen, inwiefern die Anforderungen und damit die Ziele der Arbeit erfüllt wurden.

Da die Umsetzung des Interpreters nicht im Rahmen dieser Arbeit realisiert wurde, wird insbesondere der Fokus auf die Konformität des Compilers zu RFC-9535 gesetzt und es werden die Stellen hervorgehoben, die dieser nicht entsprechen.

6.1. Erreichte Ergebnisse anhand Anforderungsliste

6.1.1. JSONPath-Funktionsumfang

Aus der Aufführung des JSONPath-Funktionsumfangs in Anforderungspunkt 1 werden alle Funktionalitäten von JSONPath durch den Compiler zumindest teilweise abgedeckt.

Der Compiler erkennt wohlgeformte JSONPath-Ausdrücke in Klammer- und Punktnotation inklusive aller Selektoren.

Der Compiler beachtet außerdem sowohl die Möglichkeiten, an denen der Ausdruck Whitespaces haben kann, als auch die meisten Stellen, an denen dies nicht der Fall ist. Stellen, an denen vom RFC auf Groß- beziehungsweise Kleinschreibung Wert gelegt wird, werden durch den Compiler durchgesetzt.

Der Compiler kann außerdem normalisierte JSONPath-Ausdrücke als solche erkennen und von allgemeinen JSONPath-Ausdrücken unterscheiden.

Im Bezug auf Test- beziehungsweise Vergleichsausdrücke stellt die Mehrdeutigkeit zwischen singulären Abfragen und allgemeinen JSONPath-Ausdrücken ein nicht vollständig gelöstes Problem dar, da der Compiler zwar im GLR-Modus diese Mehrdeutigkeit auflösen kann, aber in diesem Modus keine Syntaxbäume generieren kann, sodass zwischen vollständiger Parsebarkeit und möglicher Weiterverarbeitung entschieden werden muss.

Die Beachtung von Whitespaces kann nicht vollständig im Scanner realisiert werden, sodass das Whitespace-Verbot innerhalb von singulären Ausdrücken nicht umgesetzt werden kann. Außerdem werden Whitespaces vor dem Wurzel-Identifizierer und nach dem letzten Segment nicht beachtet, auch wenn diese je nach Interpretation des Umfangs der Ausdrücke nicht erlaubt sein sollten.

Insbesondere die Fehlerbehandlung ist ausbaufähig. Der Compiler verwirft nicht wohlgeformte Ausdrücke, liefert jedoch hierfür keine Begründungen oder hilfreiche Fehlermeldungen. Die Ergänzung eines Moduls zur ausführlichen Fehlerbehandlung könnte das System erheblich nutzerfreundlicher machen.

6.1.2. JSONPath-Compiler

Die in Anforderungspunkt 2 spezifisch für den Compiler definierten Anforderungen werden durch diesen zu großen Teilen erfüllt.

Die Kompetenzverteilung zwischen Scanner und Parser sorgt für ein möglichst präzises Ergebnis und reduziert mögliche Fehlerquellen. Der Scanner entlastet den Parser durch reguläre Ausdrücke und die Auslagerung von Whitespaces.

Der Scanner orientiert sich in seinen regulären Ausdrücken an den Grammatikregeln des RFCs, sodass dessen genaue zeichenweise Definitionen möglichst unverändert umgesetzt werden. Der Scanner kann jedoch nur 7-Bit-ASCII-Zeichen abbilden, wodurch sich der Namensraum der akzeptierten JSONPath-Ausdrücke verringert.

Der Parser überprüft die Wohlgeformtheit der Eingabe anhand der vom Scanner gelieferten Tokens. Hierfür werden die Grammatikregeln des RFCs möglichst unverändert umgesetzt, um diese nachvollziehbar und wartbar zu halten. Grammatikumformungen werden durch einheitliche Umformungsschemata realisiert.

Durch die Einbindung der Bison-Syntaxtree-Hack-Klasse in den Parser werden während des Parsevorgangs automatisiert Syntaxbäume als Zwischencode für die weitere Auswertung erzeugt.

Eine Nachbearbeitung der Syntaxbäume, beispielsweise um diese zu vereinfachen oder die bereits genannten Whitespace-Problemstellen zu beheben, erfolgt aktuell nicht.

6.1.3. JSONPath-Interpreter

Im Rahmen dieser Arbeit wurde kein JSONPath-Interpreter umgesetzt. Die Möglichkeit einer Umsetzung des Interpreters in zukünftigen Arbeiten wird durch die klare Trennung der Verantwortungsbereiche der Systemkomponenten und die Syntaxbäume als Zwischencode gewährleistet.

6.1.4. Beschleunigen der Suche in YottaDB

Wie auch der JSONPath-Interpreter, wurde die Beschleunigung der Suche in YottaDB im Rahmen dieser Arbeit nicht vorgenommen. Es wurde ein Schema zur Bewertung möglicher Algorithmen zur Beschleunigung der Suche erstellt und exemplarisch ein möglicher Algorithmus als Kandidat hierfür betrachtet und auf Laufzeit und Speicherkomplexität untersucht, jedoch wurde dieser Algorithmus unter der Annahme einer logarithmischen Laufzeit als Ausgangspunkt als nicht geeignet bewertet.

6.2. RFC-Konformität

Da die Konformität zu RFC-9535 eine starke Priorität dieser Arbeit ist, werden die Stellen, an denen der Compiler vom RFC abweicht, hier erneut hervorgehoben.

Die Umsetzung der Grammatik im Compiler ist inhaltlich vollständig, jedoch können aufgrund der Mehrdeutigkeit zwischen singulären Abfragen und allgemeinen JSONPath-Abfragen nicht alle Ausdrücke korrekt bewertet werden, solange der Syntaxbaum erstellt werden soll.

Der vom RFC vorgegebene Unicode-Zeichensatz kann vom Scanner nicht verarbeitet werden, wodurch sich der verfügbare Namensraum für JSONPath-Ausdrücke, die vom Compiler akzeptiert werden, auf den 7-Bit-ASCII-Zeichenraum reduziert.

Obgleich Whitespaces in den Klammern von singulären Ausdrücken nicht vorkommen dürfen, hat der Compiler aktuell keinen Weg, diese zu erkennen und lässt dementsprechend fehlerhafte Ausdrücke durch.

Die Fehlerverarbeitung wird im RFC explizit als wichtige Funktion einer JSONPath-Implementation hervorgehoben, jedoch wird diese bisher nur sehr oberflächlich abgehandelt.

Der Compiler überprüft außerdem Ausdrücke nur auf deren Wohlgeformtheit. Eine Überprüfung der Validität der Ausdrücke, also eine Überprüfung der Einhaltung der Integerbereiche und der Wohltypisiertheit der verwendeten Funktionserweiterungen, wird nicht umgesetzt.

6.3. Automatisierte Tests

Die automatisierten Compiler-Tests decken oberflächlich einen Großteil des JSON-Path Funktionsumfangs ab und testen explizit auf einige mögliche Fehlerquellen. Insbesondere Stellen, an denen Whitespaces nicht erlaubt sind, werden ausführlich getestet.

Die Erweiterung der Testfälle für mehr spezifische Fälle kann das Vertrauen in den Compiler weiter erhöhen, beziehungsweise eventuelle Implementationsfehler aufzeigen. Besonders ausbaufähig sind hierbei negative Testfälle, da diese in den aktuellen Tests unterrepräsentiert werden.

Ein möglicher Ansatz zur Erstellung weiterer Testfälle ist es, anhand der Grammatik im RFC für jede Regel und jede Option dieser Regel sowohl positive als auch negative Testfälle zu schreiben. Eine vollständige Abdeckung aller möglichen Testfälle beziehungsweise Testszenarien ist zwar nicht möglich, jedoch stärkt jedes weitere abgedeckte Szenario das Vertrauen in den Compiler - sofern dieser den Test erfolgreich abschließen kann.

6.4. Performance

Es wurden keine expliziten Messungen zur Laufzeit oder Speicherkomplexität des Compilers durchgeführt. Der Compile-Vorgang läuft für kleine und mittelgroße JSONPath-Ausdrücke in akzeptabler Zeit, kann jedoch bei Bedarf weiter beschleunigt werden, indem auf die Erkennung beziehungsweise Differenzierung von normalisierten Ausdrücken verzichtet wird, da für diese der Compile-Vorgang im schlimmsten Fall doppelt ausgeführt wird.

7. Zusammenfassung und Ausblick

Die Ergebnisse dieser Arbeit werden hier abschließend nochmals zusammengefasst und es wird ein Ausblick auf die mögliche Erweiterbarkeit dieser Ergebnisse gegeben.

7.1. Erreichte Ergebnisse

In dieser Arbeit wurde als Teil des Primärziels P ein Compiler für JSONPath-Ausdrücke nach RFC-9535 mit flex und Bison implementiert. Dieser orientiert sich stark an den Definitionen des RFCs, um sowohl möglichst konform zu diesem zu bleiben als auch möglichst nachvollziehbar und einfach zu warten beziehungsweise zu erweitern zu sein.

Die Sekundärziele S1 und S2 sind größtenteils erfolgreich umgesetzt worden, mit der Ausnahme der in Abschnitt 6.2 beschriebenen Probleme.

Sekundärziel S3 ist durch die Einbindung der Bison-Syntaxtree-Hack-Klasse automatisiert umgesetzt worden, sodass ein erfolgreicher Parsevorgang einen für die weitere Auswertung geeigneten Syntaxbaum als Ergebnis hat.

Die zweite Hälfte des Primärziels in Form eines Interpreters und damit zusammenhängend die Sekundärziele S4 und S5 wurden nicht umgesetzt, jedoch konnte eine theoretische Grundlage für eine zukünftige Umsetzung dieser Ziele geschaffen werden.

Die in Sekundärziel S6 beschriebene Erstellung einer komponentenbasierten Systemarchitektur in Hinsicht auf zukünftige Erweiterungen wurde umgesetzt, sodass eine Umsetzung der Interpreter-Komponenten vergleichsweise reibungslos möglich ist.

7.2. Ausblick

Ein mögliches Hauptthema einer zukünftigen Erweiterung dieser Ergebnisse wäre entsprechend des vorausgehenden Abschnitts die Umsetzung der Interpreter-Komponenten, um die überprüften JSONPath-Ausdrücke gegen einen beispielsweise in YottaDB gespeicherten Datensatz auszuwerten.

Durch die Ergänzung einer ausführlichen Fehlerverarbeitung kann das System erheblich nutzerfreundlicher gestaltet werden. Ebenso könnte das System je nach Einsatzort um ein User Interface erweitert werden, um es für mehr Nutzer zugänglich zu machen.

Für die Vervollständigung der RFC-Konformität könnte der Scanner auf geeignete Weise erweitert werden, sodass er alle Unicode-Zeichen verarbeiten kann. Außerdem können Lösungen für die noch ungeklärten Whitespaces in singulären Abfragen und die Mehrdeutigkeit zwischen singulären und allgemeinen Abfragen gesucht werden.

Um die Suche in YottaDB beschleunigen zu können, können weitere Algorithmen anhand der erstellten Kriterien auf Laufzeit und Speicherkomplexität überprüft werden. Außerdem kann die Beschleunigung von der direkten Schlüsselsuche auf die Suche über reguläre Ausdrücke nach einem Teilschlüssel erweitert werden, an welcher Stelle wahrscheinlich auf ein größeres Verbesserungspotenzial gestoßen werden kann.

Der in dieser Arbeit erstellte Compiler erzeugt einen JSONPath-Syntaxbaum, welcher als Ausgangspunkt verwendet werden kann, um im nächsten Schritt eine spezifische Datenbank für die Auswertung anzusprechen.

Literatur

- [1] Prof. Dr. Winfried Bantel. *Bison Syntaxtree Hack*. 2024. URL: <https://github.com/informatik-aalen/Bison-Syntaxtree-Hack> (besucht am 26.05.2026).
- [2] Prof. Dr. Winfried Bantel. „Compilerbau“. 2022.
- [3] Will Estes. *flex*. URL: <https://github.com/westes/flex/> (besucht am 26.05.2026).
- [4] Inc. Free Software Foundation. *Bison 3.8.1*. 2021. URL: <https://www.gnu.org/software/bison/manual/bison.html> (besucht am 26.05.2026).
- [5] Inc. Free Software Foundation. *Writing GLR Parsers1*. 2021. URL: https://www.gnu.org/software/bison/manual/html_node/GLR-Parsers.html (besucht am 26.05.2026).
- [6] Helmut Herold. *lex & yacc: Die Profitools zur lexikalischen und syntaktischen Textanalyse*. Addison-Wesley, 2003. ISBN: 3-8273-2096-8.
- [7] Vern Paxson. *flex - fast lexical analyzer generator*. URL: <https://web.stanford.edu/class/archive/cs/cs143/cs143.1112/materials/other/manflex.html> (besucht am 26.05.2026).
- [8] Ed. S. Gössner. *RFC 9535: JSONPath: Query Expressions for JSON*. 2024. URL: <https://www.rfc-editor.org/rfc/rfc9535.html> (besucht am 26.05.2026).
- [9] Lauren Schaefer. *Was ist NoSQL?* URL: <https://www.mongodb.com/de-de/resources/basics/databases/nosql-explained> (besucht am 26.05.2026).
- [10] Ed. T. Bray. *RFC 8259: STD 90: The JavaScript Object Notation (JSON) Data Interchange Format*. 2017. URL: <https://www.rfc-editor.org/info/rfc8259/> (besucht am 26.05.2026).
- [11] YottaDB. *About Us*. URL: <https://yottadb.com/about-us/> (besucht am 26.05.2026).
- [12] YottaDB. *How It Works*. URL: <https://yottadb.com/product/how-it-works/> (besucht am 26.05.2026).
- [13] YottaDB. *YottaDB*. URL: <https://yottadb.com/> (besucht am 26.05.2026).

A. JSONPath-Grammatik in EBNF

A.1. Grammatik-Regeln

```
1  jsonpath_query = root_identifier segments
2  segments = {S segment}
3
4  selector = name_selector
5             | wildcard_selector
6             | slice_selector
7             | index_selector
8             | filter_selector
9
10 name_selector = string_literal
11
12 index_selector = int
13
14 slice_selector = [start S] ":" S [end S] [":" [S step]]
15 start = int
16 end = int
17 step = int
18
19 filter_selector = "?" S logical_expr
20
21 logical_expr = logical_or_expr
22 logical_or_expr = logical_and_expr {S "||" S logical_and_expr}
23 logical_and_expr = basic_expr {S "&&" S basic_expr}
24
25 basic_expr = paren_expr
26             | comparison_expr
27             | test_expr
28
29 paren_expr = [logical_not_op S] "(" S logical_expr S ")"
30
31 test_expr = [logical_not_op S] (filter_query | function_expr)
32 filter_query = rel_query | jsonpath_query
33 rel_query = current_node_identifier segments
34
35 comparison_expr = comparable S comparison_op S comparable
36
37 literal = number
```

```

38         | string_literal
39         | true
40         | false
41         | null
42
43     comparable = literal
44         | singular_query
45         | function_expr
46
47     singular_query = rel_singular_query
48         | abs_singular_query
49     rel_singular_query = current_node_identifier
50         ↪ singular_query_segments
51     abs_singular_query = root_identifier singular_query_segments
52     singular_query_segments = {S (name_segment | index_segment)}
53     name_segment = ("[" name_selector "]" )
54         | ("." member_name_shorthand)
55     index_segment = "[" index_selector "]"
56
57     number = (int | "-0") [frac] [exp]
58     frac = "." DIGIT {DIGIT}
59     exp = "e" ["+" | "-"] DIGIT {DIGIT}
60
61     function_expr = function_name "(" S [function_argument
62         {S "," S function_argument}] S ")"
63
64     function_argument = literal
65         | filter_query
66         | logical_expr
67         | function_expr
68
69     segment = child_segment
70         | descendant_segment
71     child_segment = bracketed_selection
72         | ("." (wildcard_selector | member_name_shorthand))
73
74     bracketed_selection = "[" S selector {S "," S selector} S "]"
75
76     descendant_segment = ".." (bracketed_selection
77         | wildcard_selector
78         | member_name_shorthand)

```

A.2. Zeichenregeln

```

1  B = " "
2      | "\t"
3      | "\n"
4      | "\c"
5  S = {B}
6
7  root_identifier = "$"
8
9
10 string_literal = "'" {double_quoted} "'"
11                | '"' {single_quoted} '"'
12
13 double_quoted = unescaped
14                | "'"
15                | ESC "'"
16                | ESC escapable
17
18 single_quoted = unescaped
19                | '"'
20                | ESC '"'
21                | ESC escapable
22
23 ESC = "\"
24
25 unescaped = [\x20-\x21]
26            | [\x23-\x26]
27            | [\x22-\x5B]
28            | [\x5D-\xD7FF]
29            | [\xE000-\x10FFFF]
30
31 escapable = "b"
32            | "f"
33            | "n"
34            | "r"
35            | "t"
36            | "/"
37            | "\"
38            | ("u" hexchar)
39
40 hexchar = non_surrogate
41           | (high_surrogate "\u" low_surrogate)
42 non_surrogate =
43     ((DIGIT | "A" | "B" | "C" | "E" | "F") HEXDIG HEXDIG HEXDIG)
44     | ("D" [0-7] HEXDIG HEXDIG)
45 high_surrogate = "D" ("8" | "9" | "A" | "B") HEXDIG HEXDIG
46 low_surrogate = "D" ("C" | "D" | "E" | "F") HEXDIG HEXDIG
47 HEXDIG = DIGIT

```



```

48         | "A"
49         | "B"
50         | "C"
51         | "D"
52         | "E"
53         | "F"
54
55
56     wildcard_selector = "*"
57
58     int = "0"
59         | ([ "-" ] DIGIT1 {DIGIT})
60     DIGIT1 = [1-9]
61
62     logical_not_op = "!"
63
64     current_node_identifier = "@"
65
66     comparison_op = "=="
67         | "!="
68         | "<="
69         | ">="
70         | "<"
71         | ">"
72
73     true = "true"
74     false = "false"
75     null = "null"
76
77     function_name = function_name_first {function_name_char}
78     function_name_first = LCALPHA
79     function_name_char = function_name_first
80         | "_"
81         | DIGIT
82
83     LCALPHA = [a-z]
84
85     member_name_shorthand = name_first {name_char}
86     name_first = ALPHA
87         | "_"
88         | [\x80-\xD7FF]
89         | [\xE000-\x10FFFF]
90     name_char = name_first
91         | DIGIT
92
93     DIGIT = [0-9]
94     ALPHA = [A-Za-z]

```

A.3. Normalisierte Abfragen

```

1  normalized_path = root_identifier {normal_index_segment}
2  normal_index_segment = "[" normal_selector "]"
3
4  normal_selector = normal_name_selector
5                  | normal_index_selector
6
7  normal_name_selector = "'" {normal_single_quoted} "'"
8  normal_single_quoted = normal_unescaped
9                      | ESC normal_escapable
10
11 normal_unescaped = [\x20-\x26]
12                 | [\x28-\x5B]
13                 | [\x5D-\xD7FF]
14                 | [\xE000-\x10FFFF]
15
16 normal_escapable = "b"
17                 | "f"
18                 | "n"
19                 | "r"
20                 | "t"
21                 | "'"
22                 | "\"
23                 | ("u" normal_hexchar)
24
25 normal_hexchar = "00"
26                (
27                  ("0" [0-7])
28                  | ("0b")
29                  | ("0" [e-f])
30                  | ("1" normal_HEXDIG)
31                )
32
33 normal_HEXDIG = DIGIT
34               | [a-f]
35
36
37 normal_index_selector = "0"
38                     | (DIGIT1 {DIGIT})

```

B. Syntaxbäume

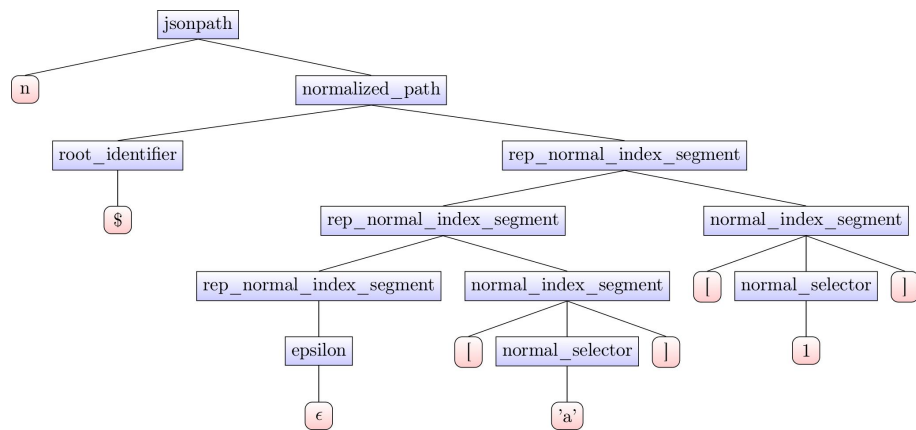


Abbildung B.1.: Syntaxbaum für Ausdruck `$['a']/[1]`

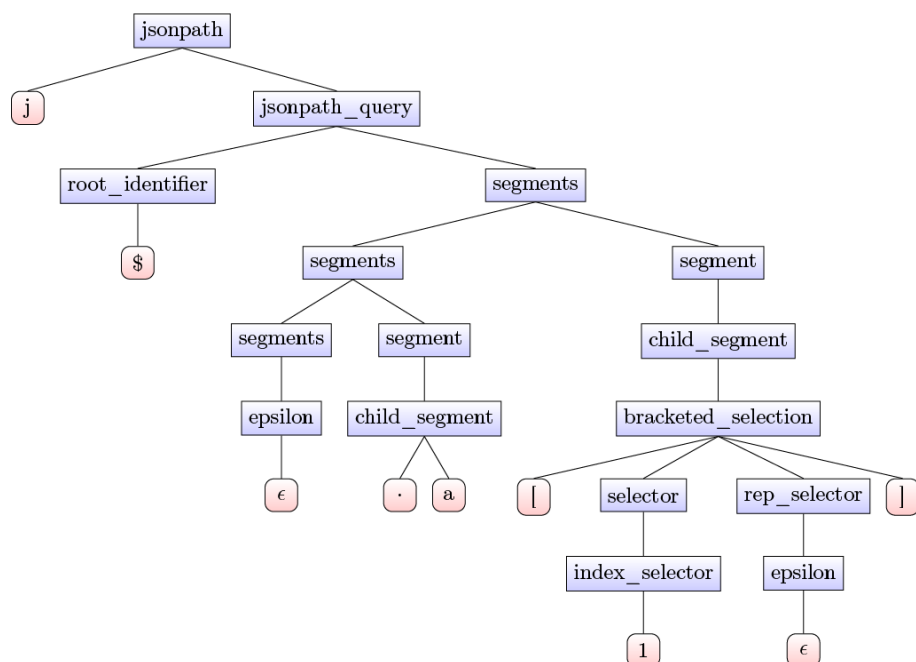


Abbildung B.2.: Syntaxbaum für Ausdruck `$.a[1]`

